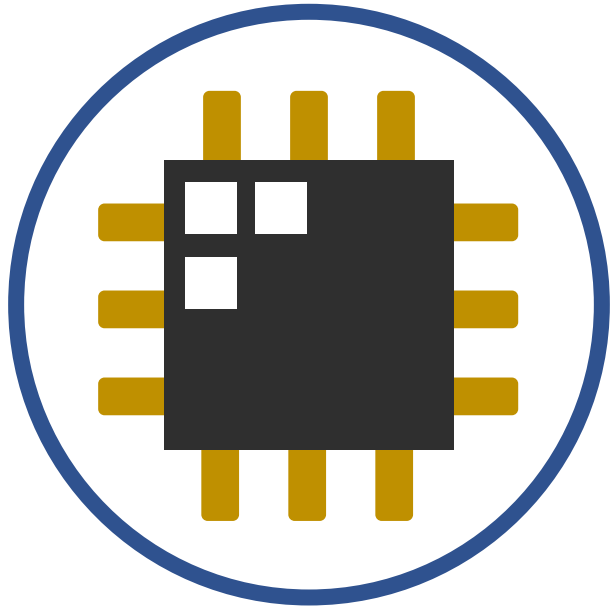
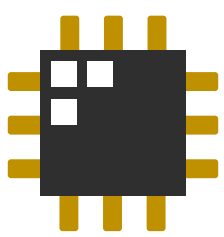




EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 4 – The computer system,
Computing Component Wrap-up



Computer systems

- *Computers* are machines that were invented to help make the “thinking work” easier and faster



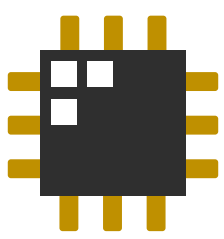
Solving equations



Tracking information

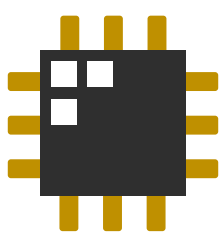


Analyzing information



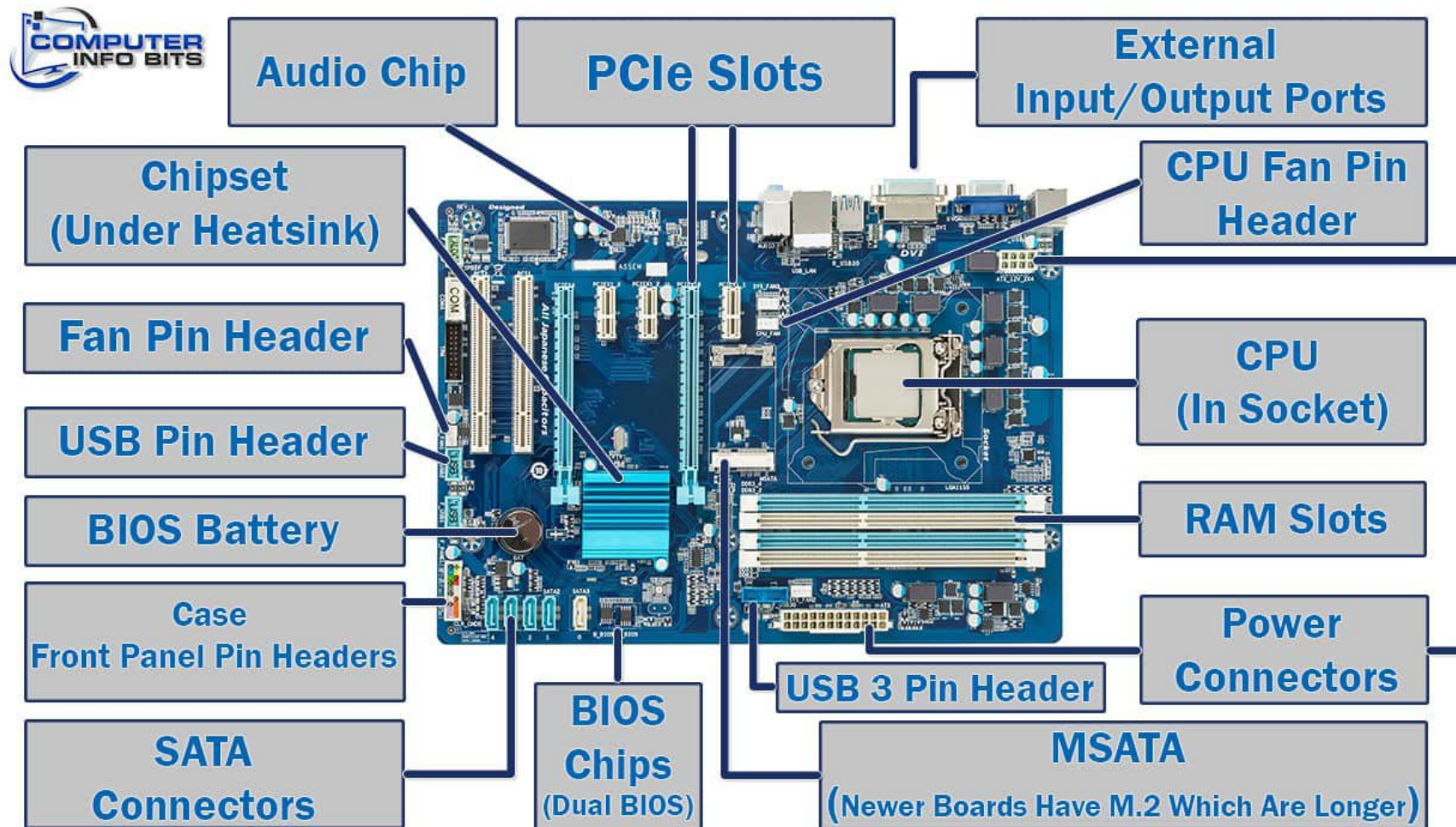
Computer systems

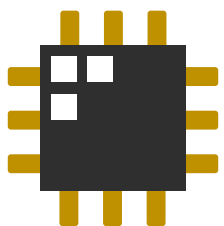




A computer system

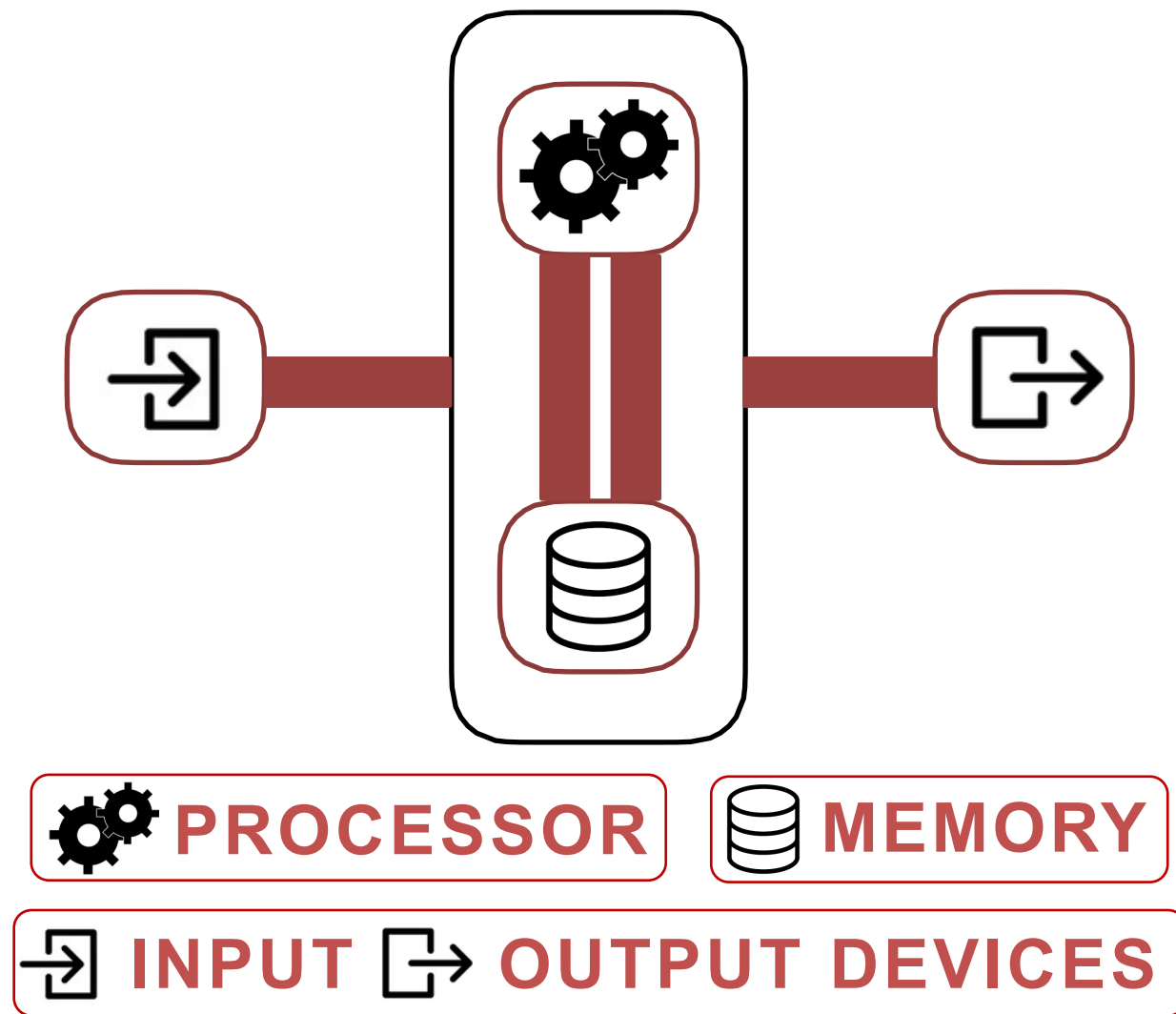
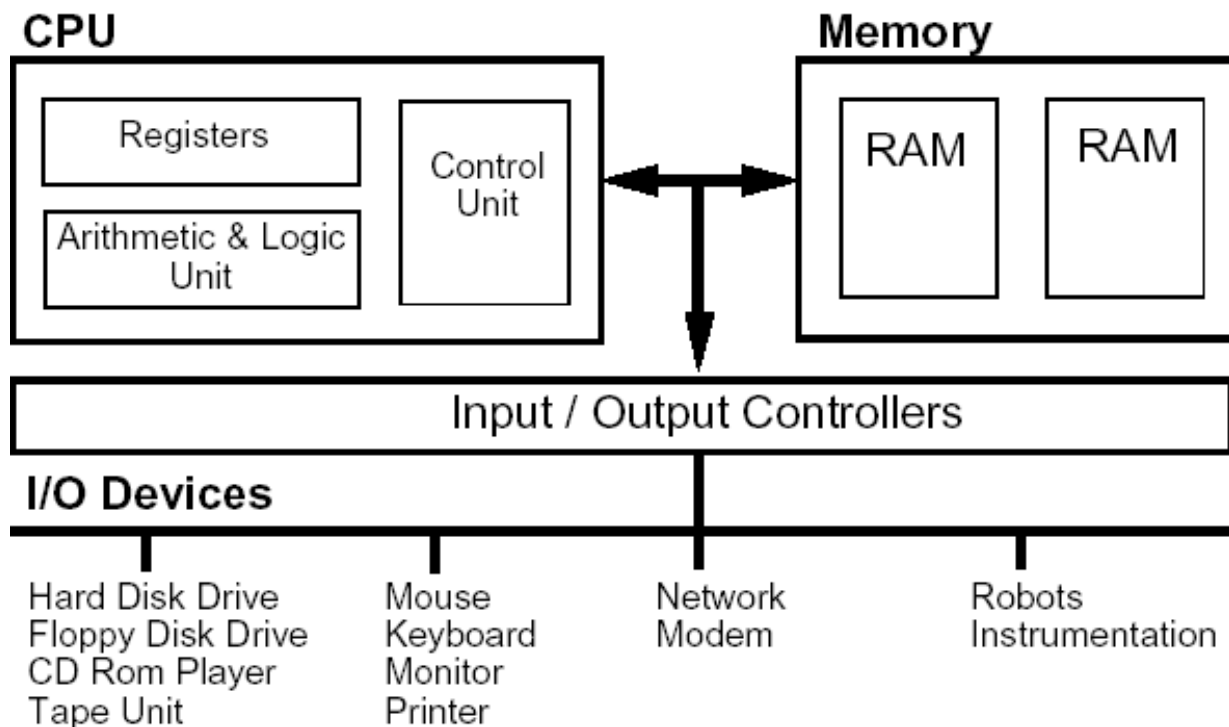
- Breaking apart a computer...

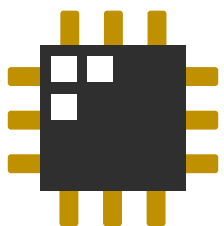




A computer system

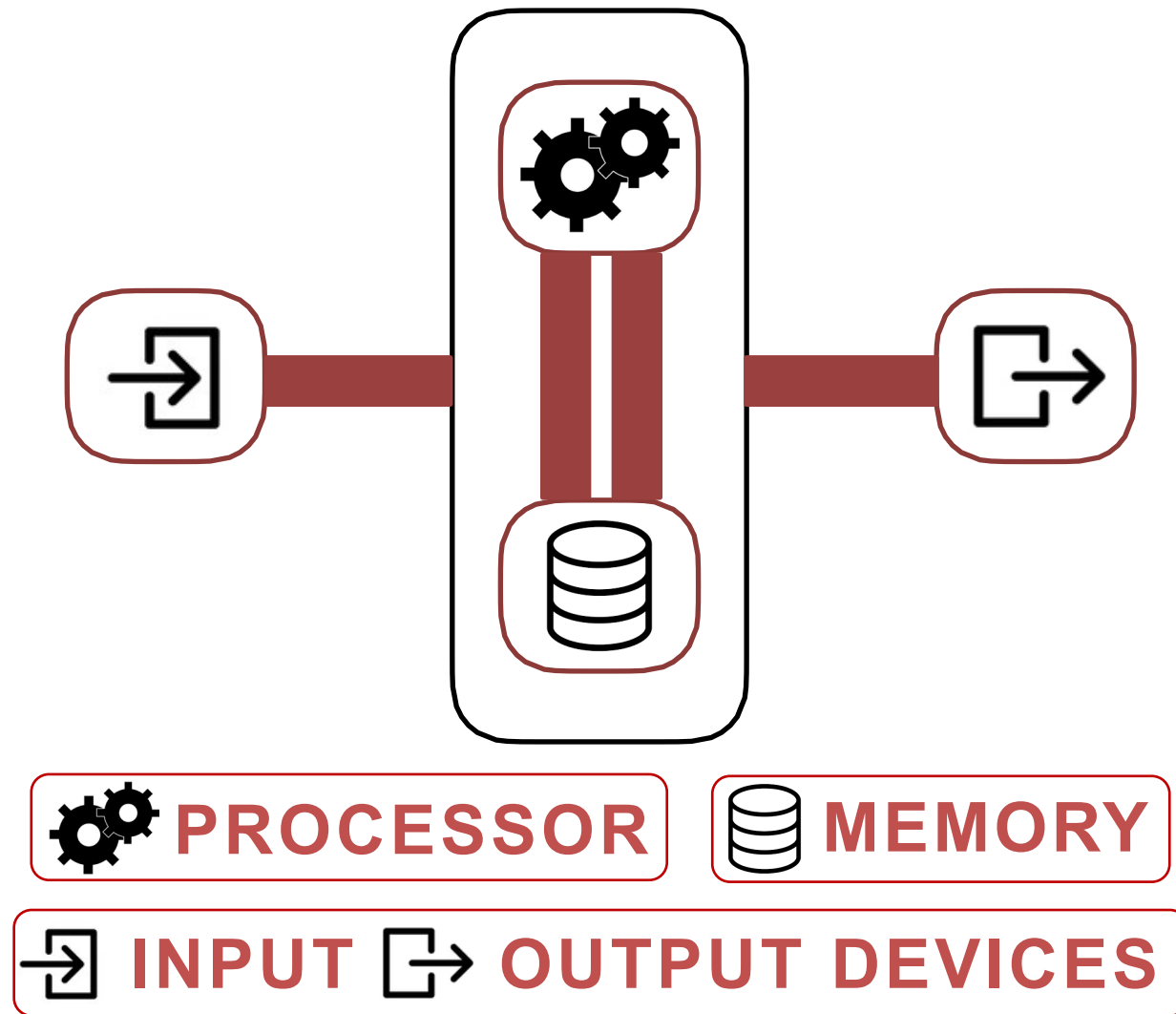
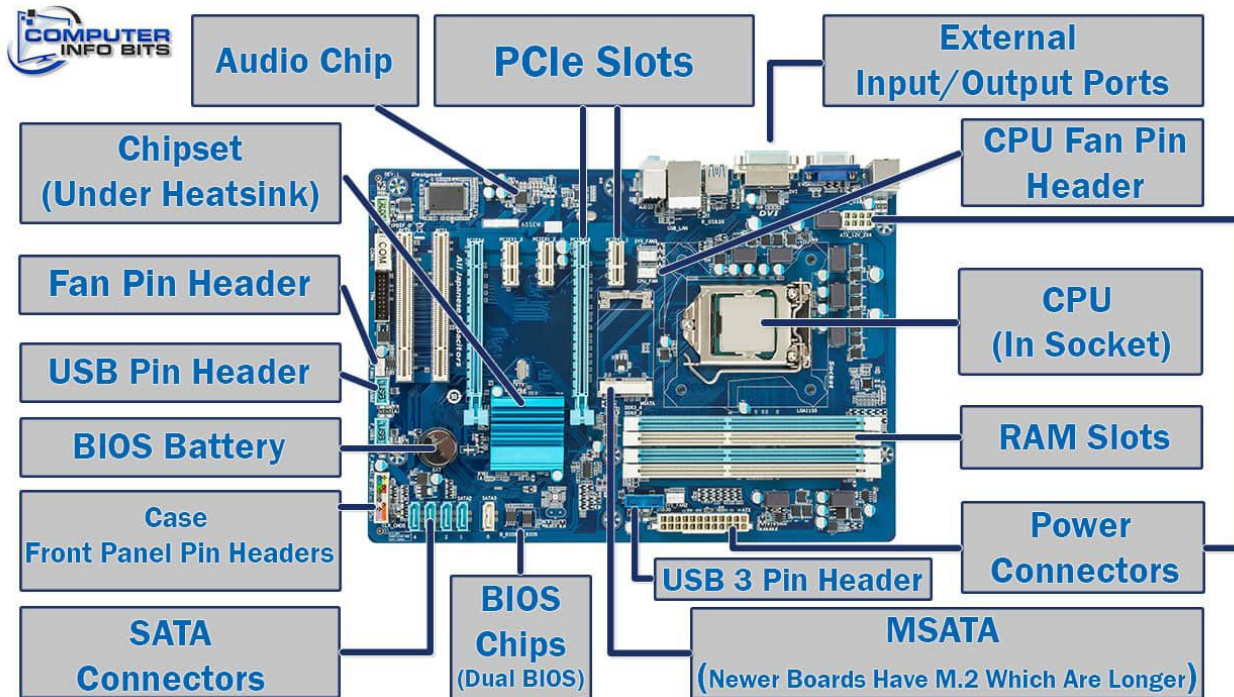
- Breaking apart a computer...

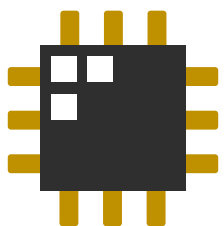




A computer system

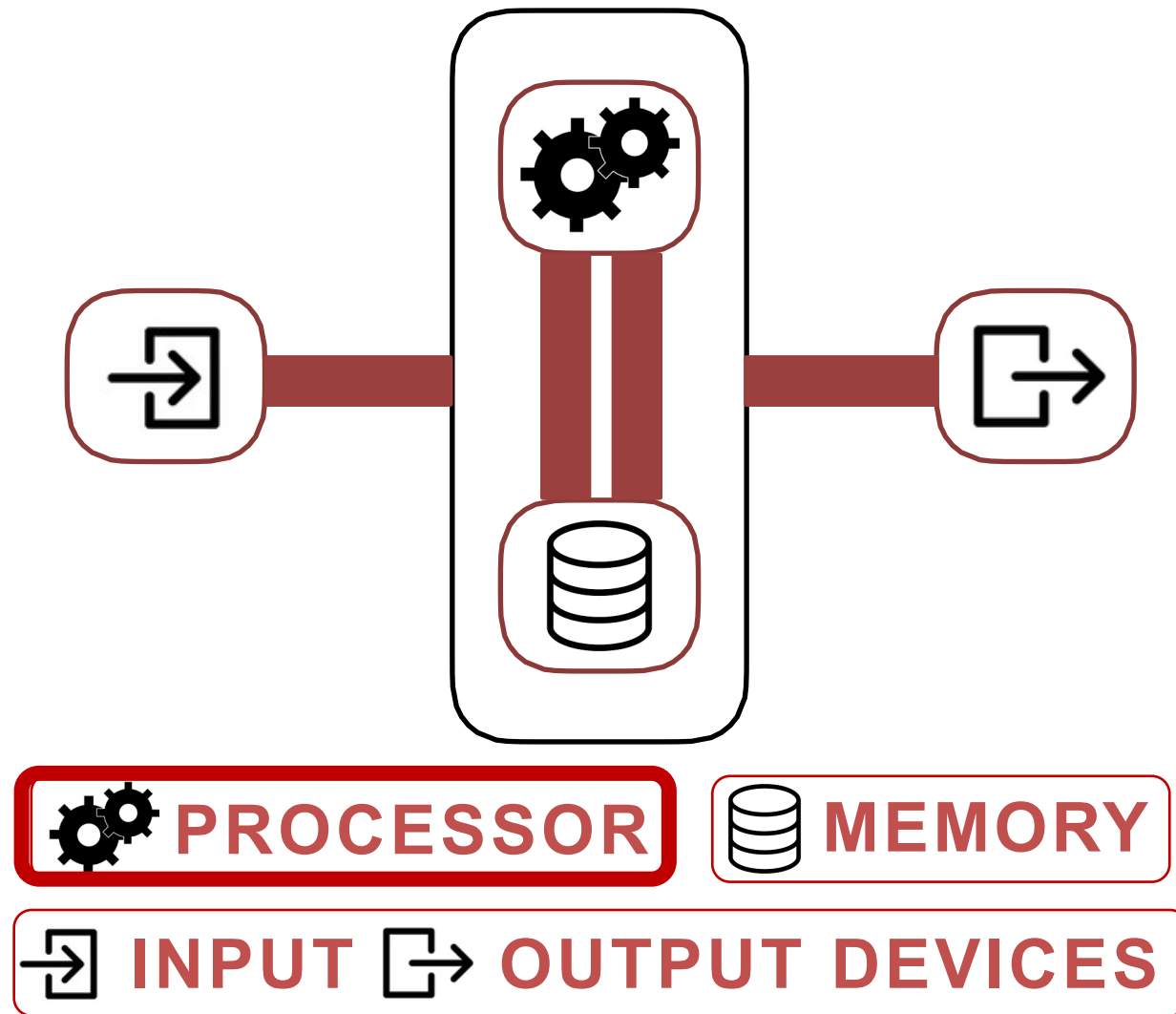
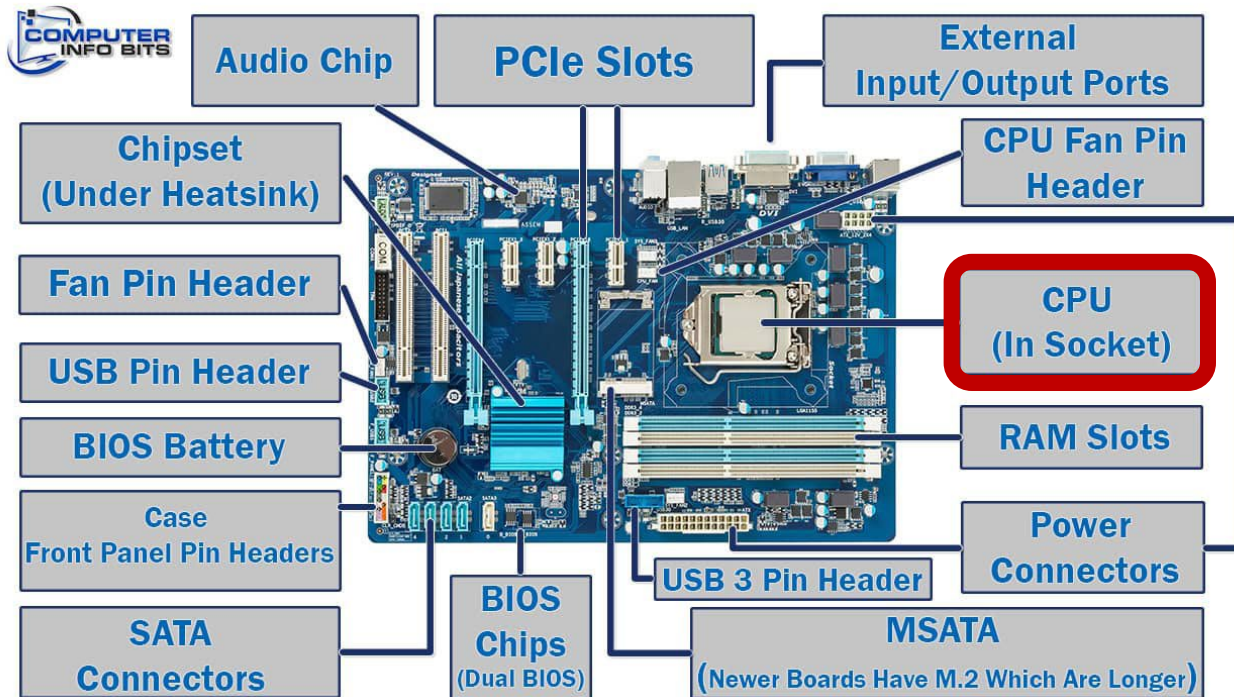
- Breaking apart a computer...

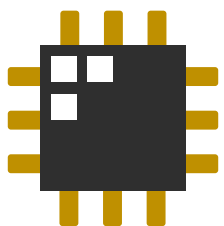




A computer system

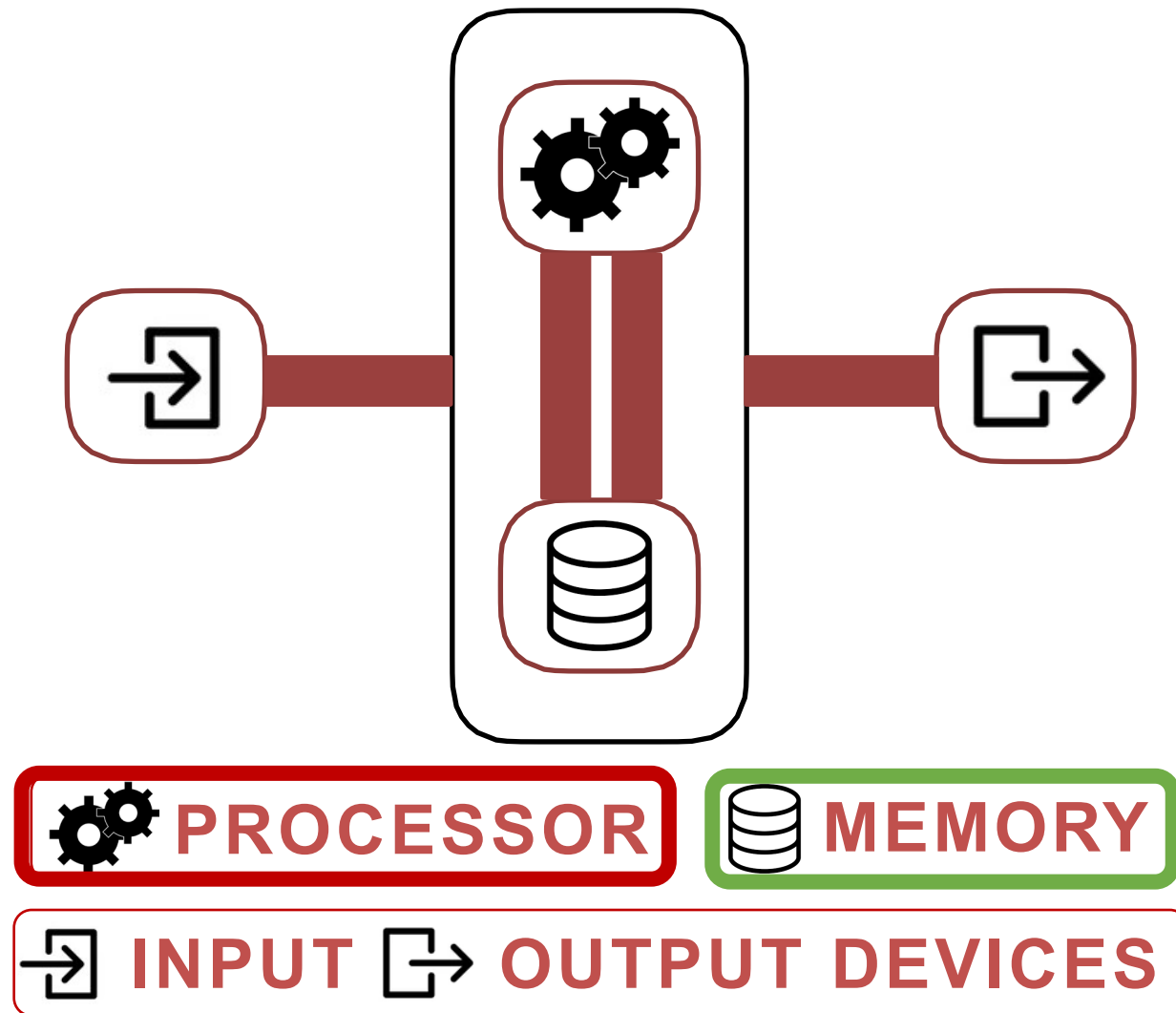
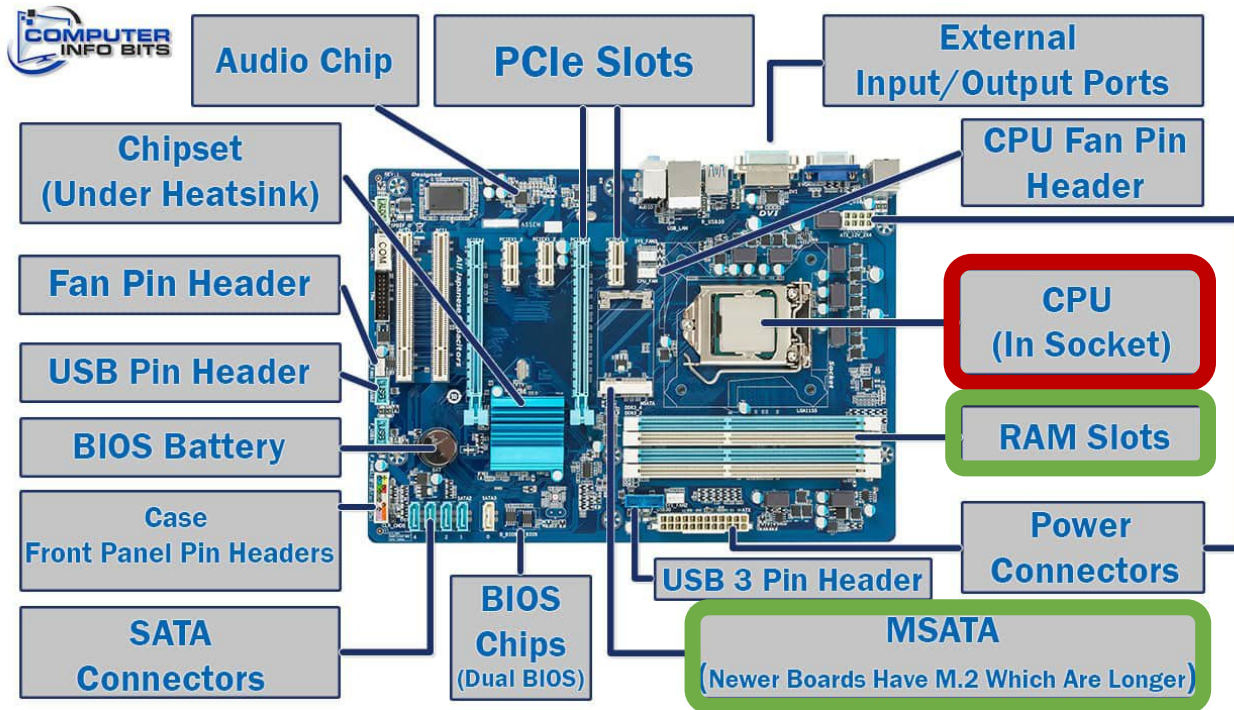
- Breaking apart a computer...

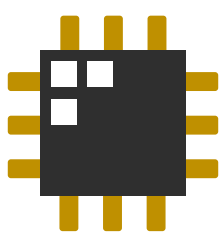




A computer system

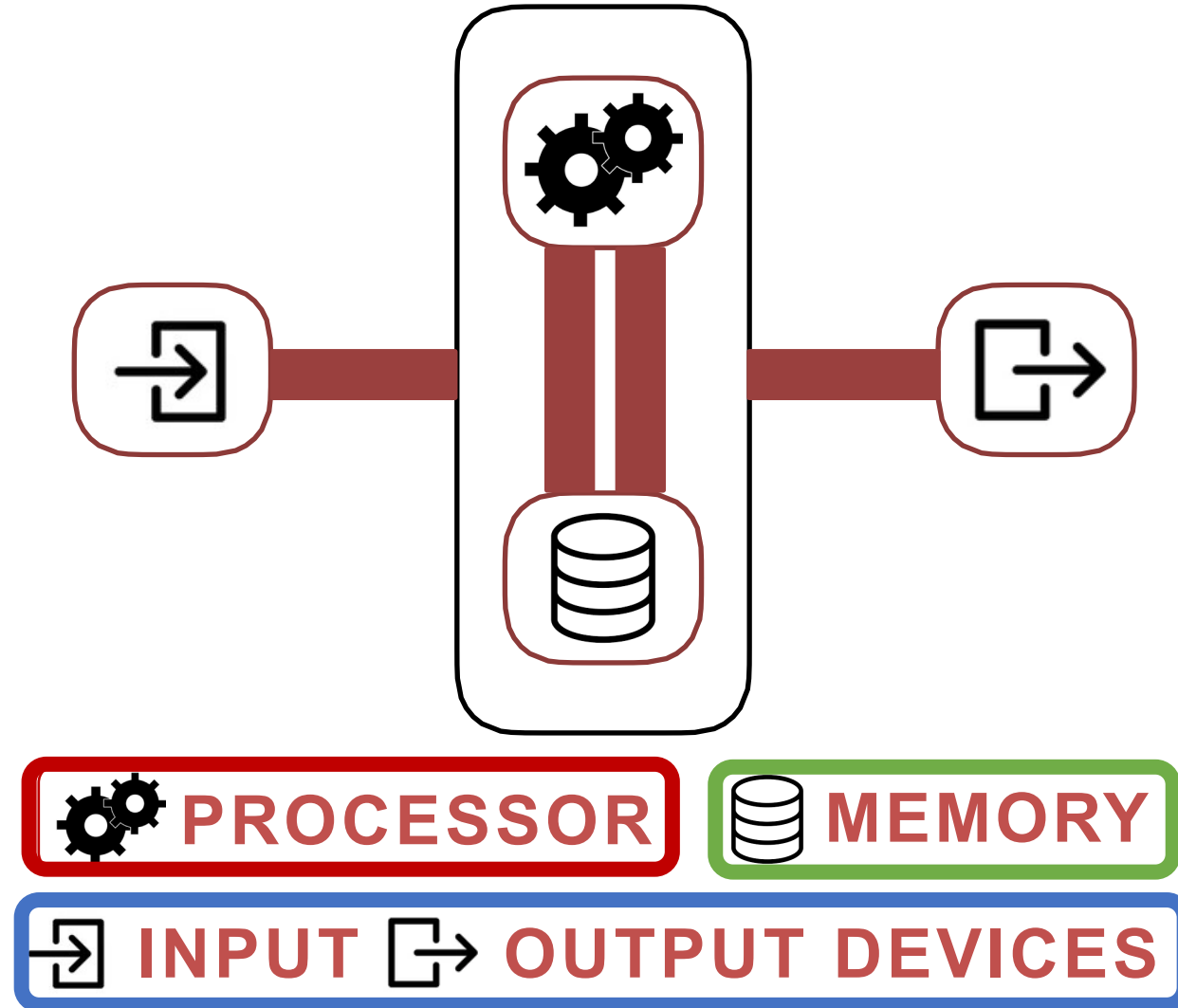
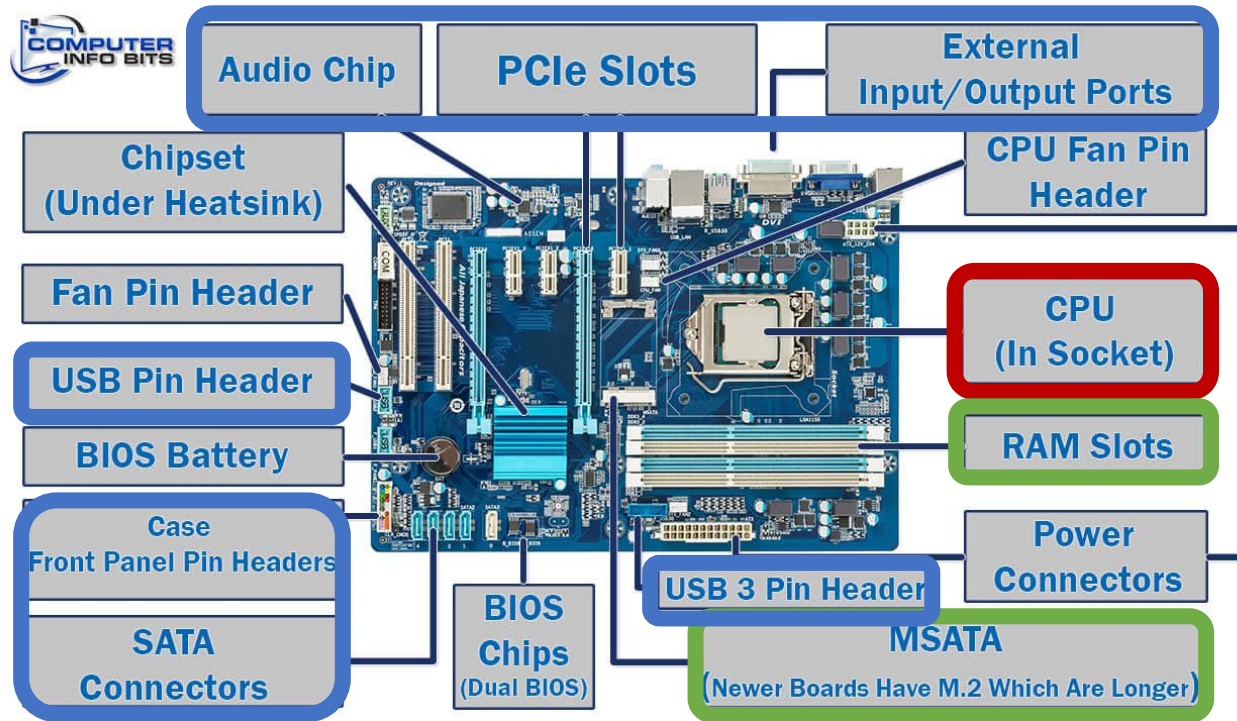
- Breaking apart a computer...

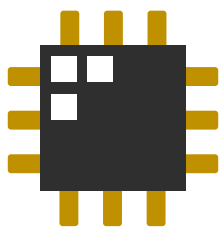




A computer system

- Breaking apart a computer...

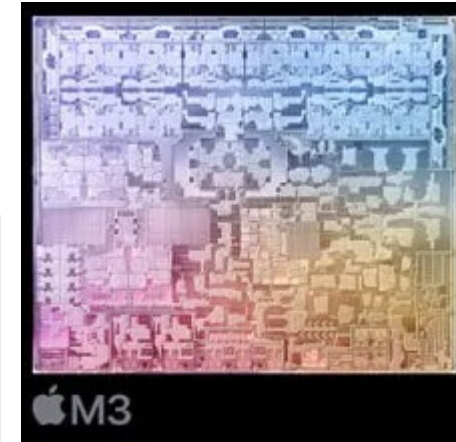
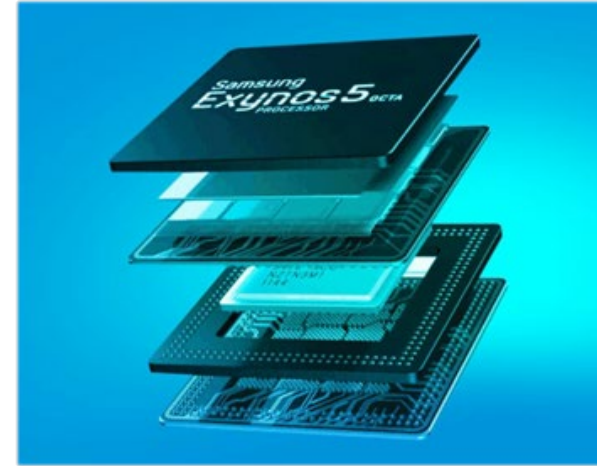
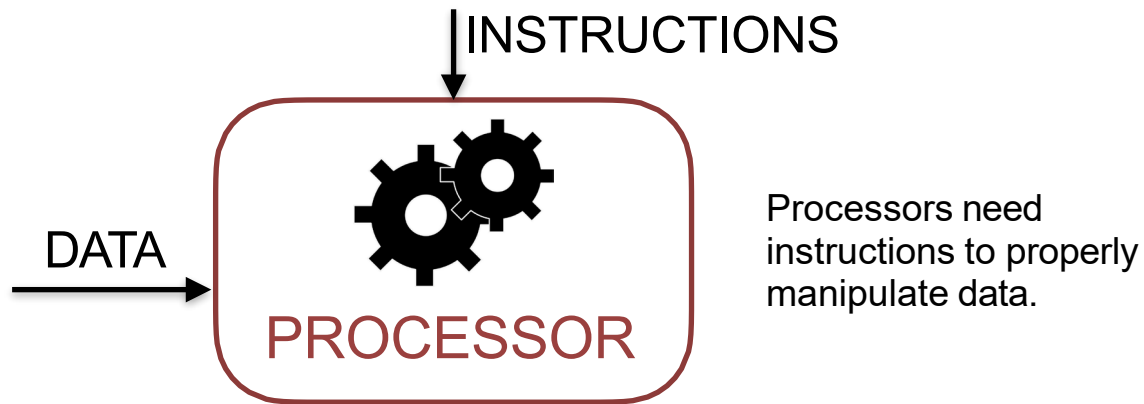


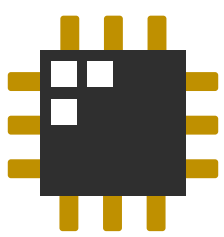


A computer system



- **Processor**, also called the Central processing unit (CPU)
 - Contains several functional units for performing operations on data
 - Has internal memory called registers to contain operands

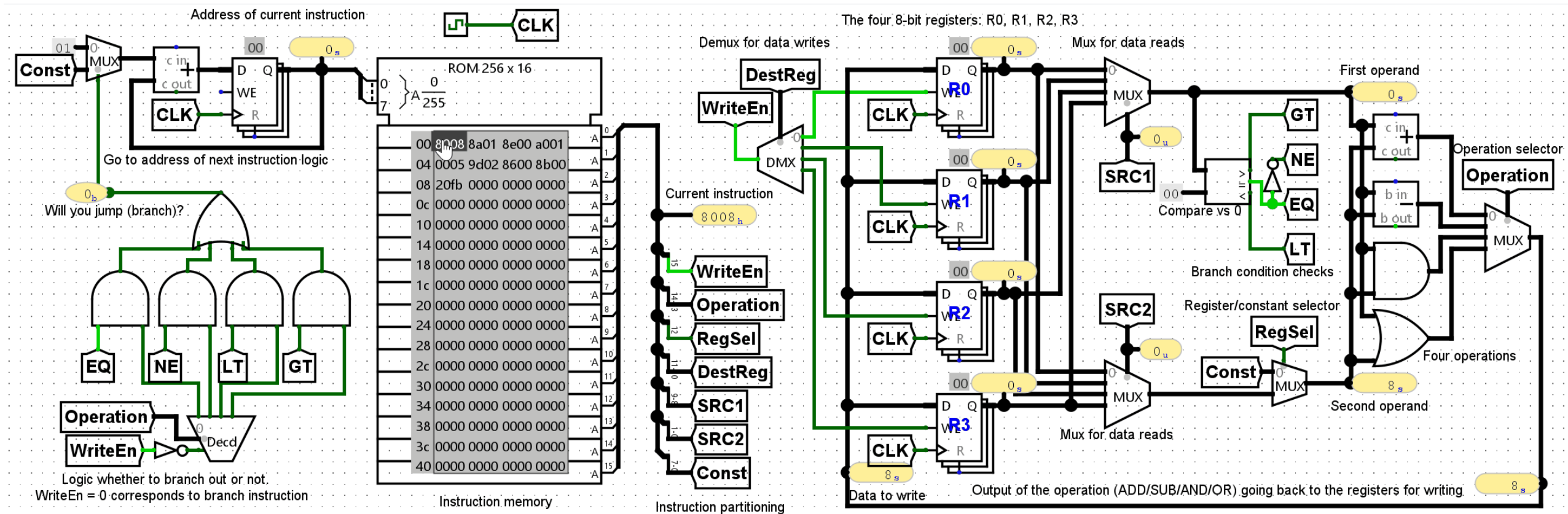


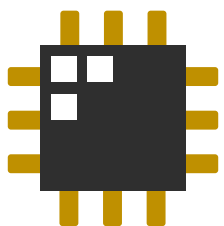


A computer system



- **Processor**, also called the Central processing unit (CPU)
 - Contains several functional units for performing operations on data
 - Has internal memory called registers to contain operands





A computer system



PROCESSOR

- **Processor**, also called the Central processing unit (CPU)
 - Contains several functional units for performing operations on data
 - Has internal memory called registers to contain operands

```
.data
a: .word 17
b: .word 9

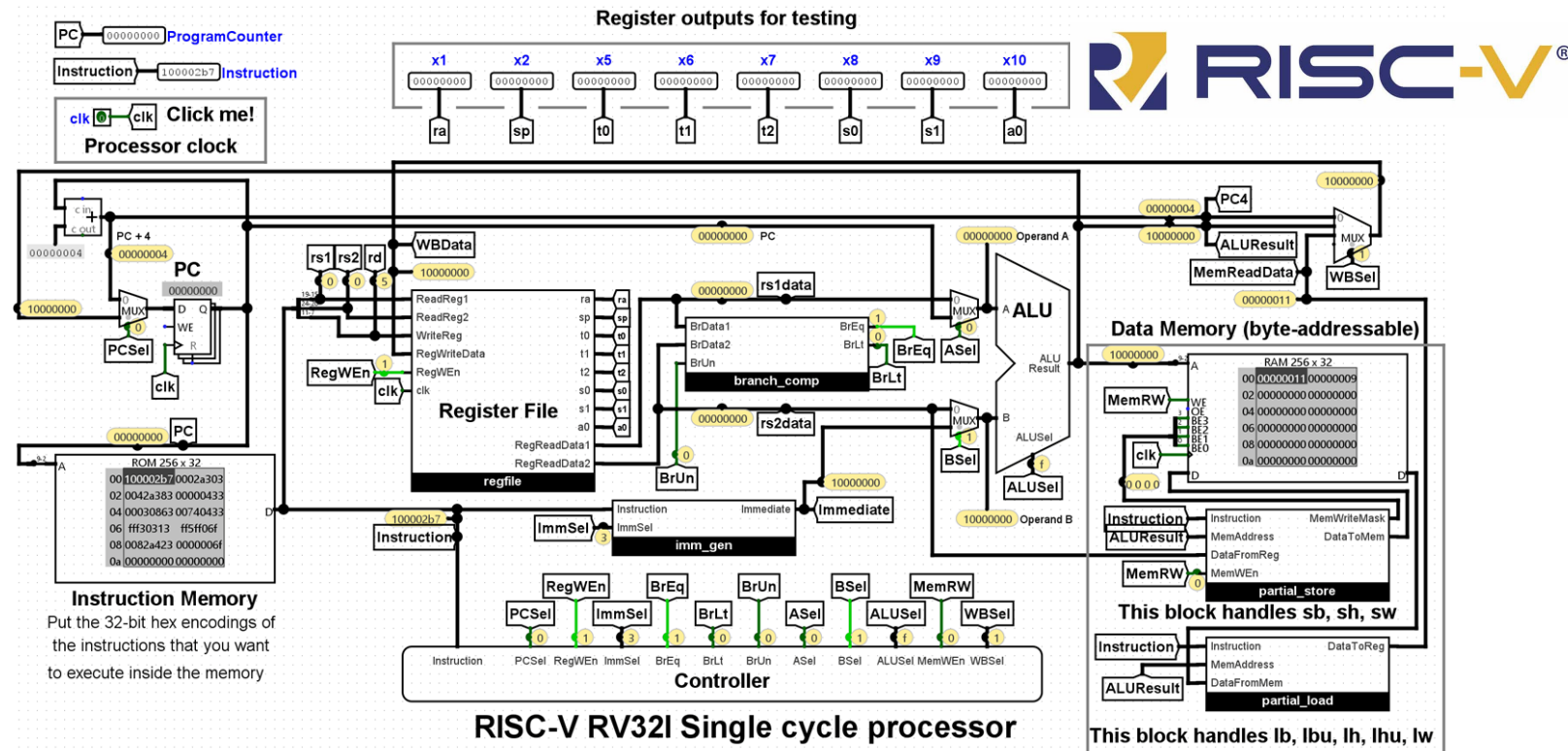
.text
lui t0, 0x10000
lw t1, 0(t0)
lw t2, 4(t0)
add s0, x0, x0
loop:
beq t1, x0, done
add s0, s0, t2
addi t1, t1, -1
j loop
done:
sw s0, 8(t0)
inf_loop: j inf_loop
```

Assembly program on
RISC-V architecture

Multiplication using
repeated addition

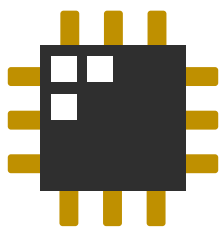
Equivalent 32-bit
binary encoding

Program Counter	Binary translation	Actual instruction
0x00000000	0x10002B7	lui x5 65536
0x00000004	0x0002A303	lw x6 0(x5)
0x00000008	0x0042A383	lw x7 4(x5)
0x0000000C	0x00000433	add x8 x0 x0
0x00000010	0x00030863	beq x6 x0 16
0x00000014	0x00740433	add x8 x8 x7
0x00000018	0xFFFF30313	addi x6 x6 -1
0x0000001C	0xFF5FF06F	jal x0 -12
0x00000020	0x0082A423	sw x8 8(x5)
0x00000024	0x0000006F	jal x0 0



Typical architecture of a 32-bit, single-cycle, single-core processor

No need to understand all of this. It is meant to be overwhelming. 😊 Watch out for this in EEE 153!

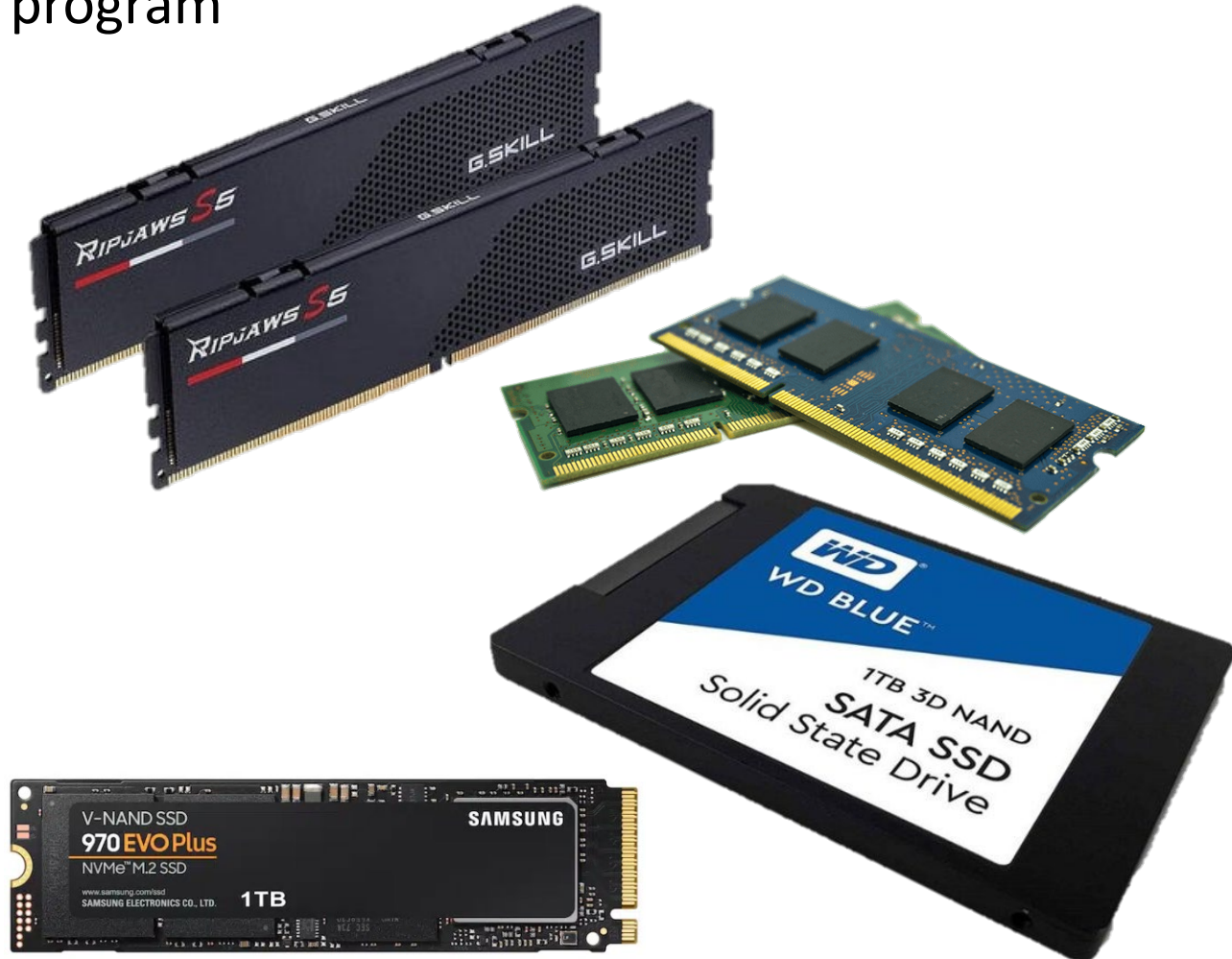
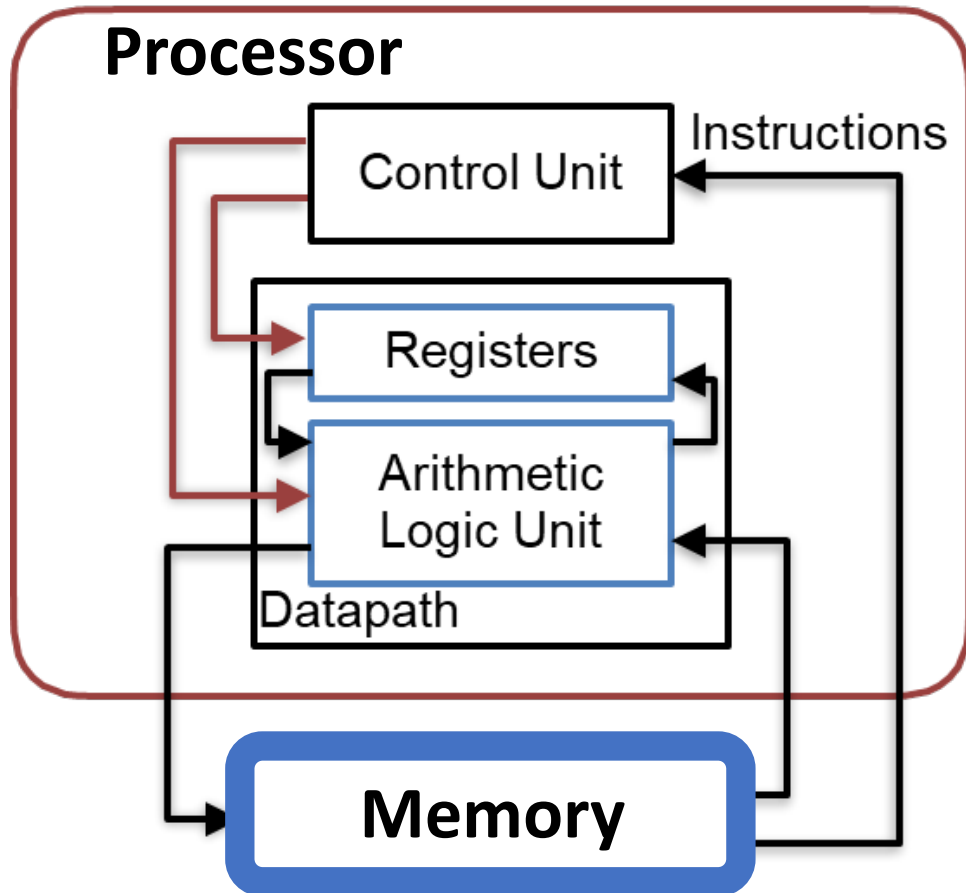


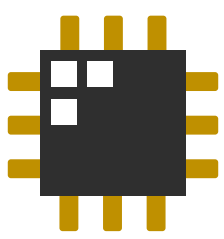
A computer system



- **Memory**

- Contains the instructions and data for the program



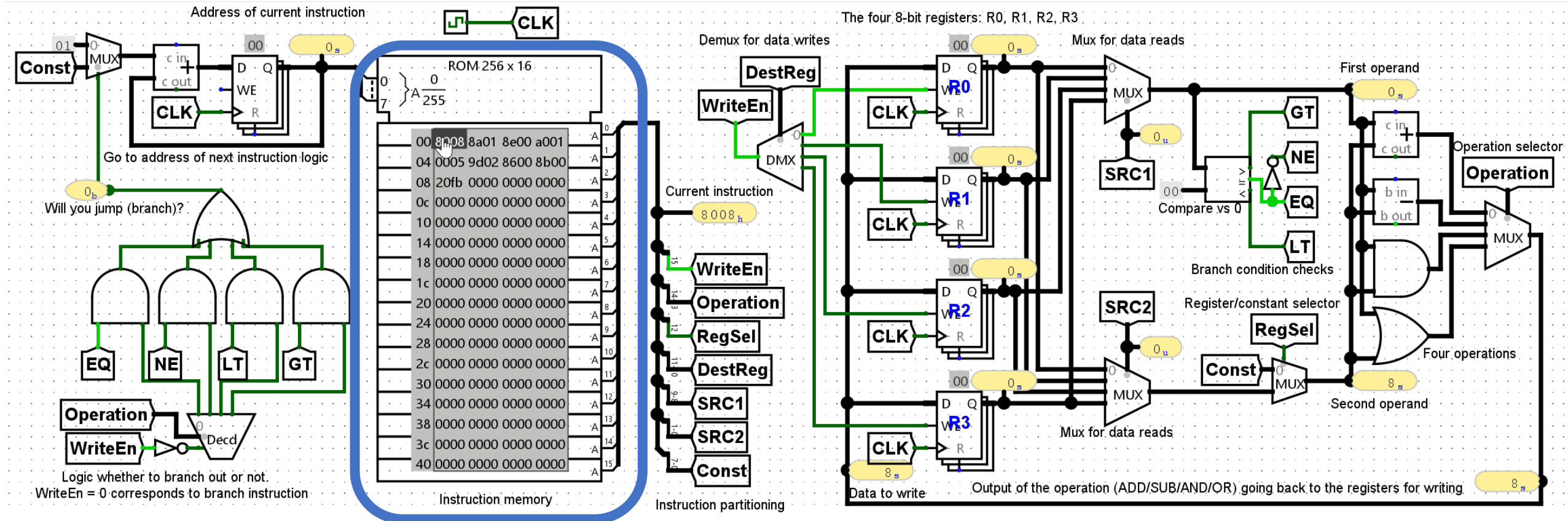


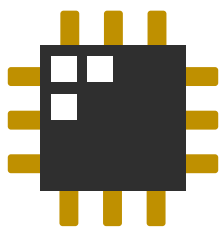
A computer system



- **Memory**

- Contains the instructions and data for the program





A computer system



- **Memory**

- Contains the instructions and data for the program
- Typically divided into two classes:
 - Main memory
 - Secondary memory



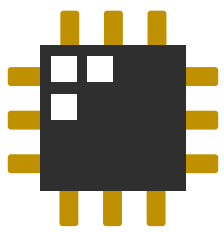
Main memory

This is just a temporary storage for data



Secondary memory

This is where programs are installed



A computer system



- **Memory**

- Contains the instructions and data for the program
- Typically divided into two classes:
 - Main memory: This is your RAM; fast but *volatile* memories

Volatile: power off, data is lost

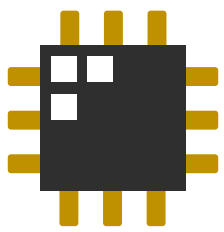


Main memory

This is just a temporary storage for data

Random Access Memory (RAM) is a type of volatile memory.

When you have unsaved work (currently in RAM) and your computer suddenly shuts down... that work is lost since RAM is volatile.



A computer system



• Memory

- Contains the instructions and data for the program
- Typically divided into two classes:
 - Main memory: This is your RAM, fast but *volatile* memories
 - Secondary memory: These are your hard drives and SSDs; slower but *non-volatile* memories

Volatile: power off, data is lost
Non-volatile: power off, data is still there

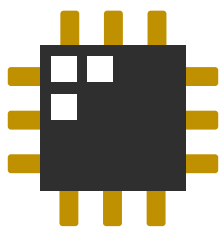
Hard disk drives (HDDs) and Solid-state drives (SSDs) are types of non-volatile memories.

When you save your work, it is committed to the secondary memory.

Removing the power is fine since it is now saved in non-volatile memory.

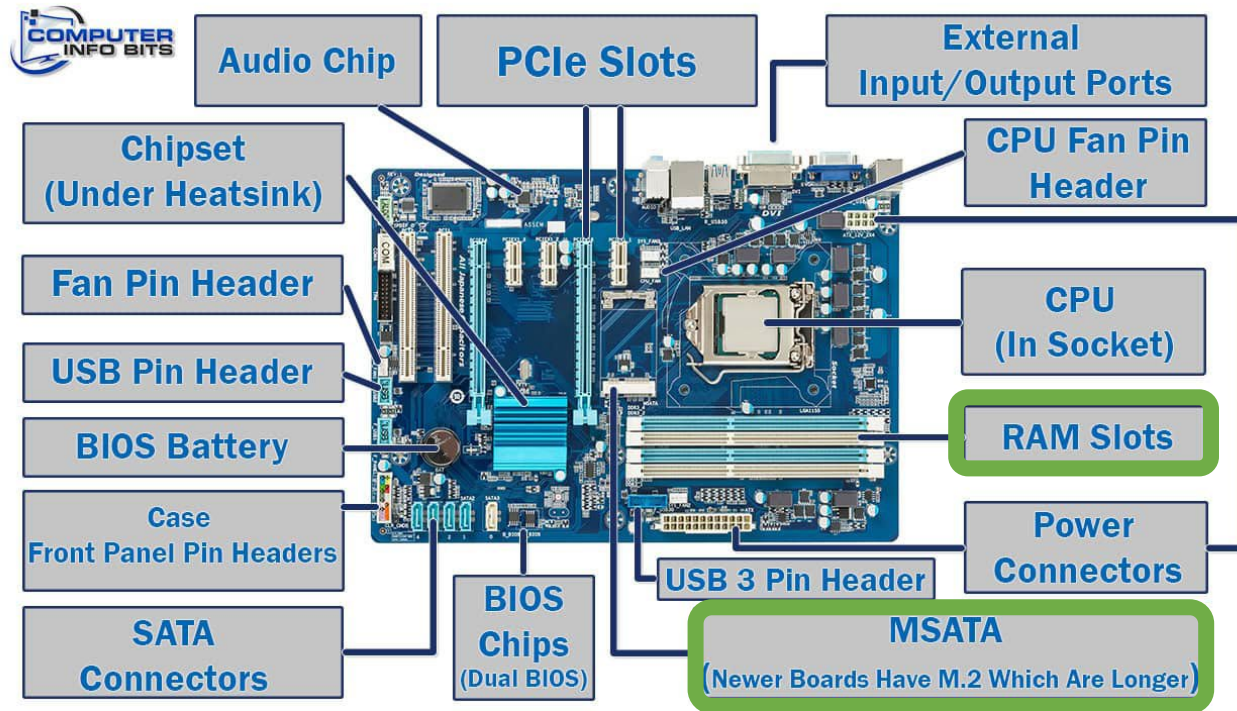


Secondary memory
This is where programs are installed

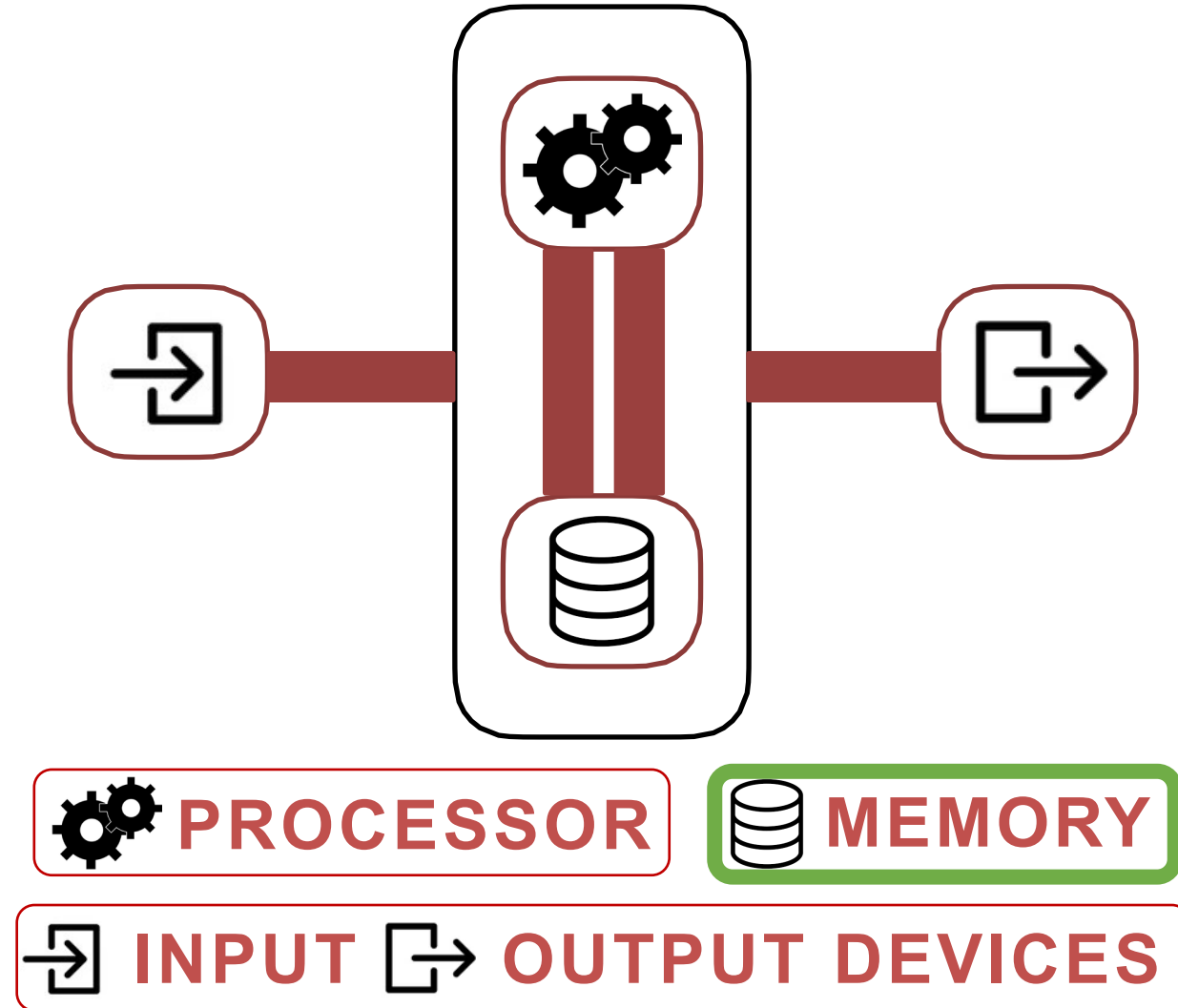


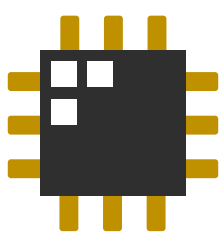
A computer system

- Breaking apart a computer...



You'll learn how the processor handles both the main and secondary memories in EEE 153!





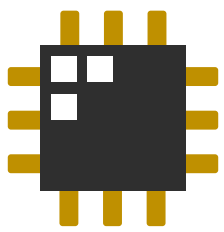
A computer system

➡ **INPUT** ➡ **OUTPUT DEVICES**

- **Input and Output (I/O) interfaces**

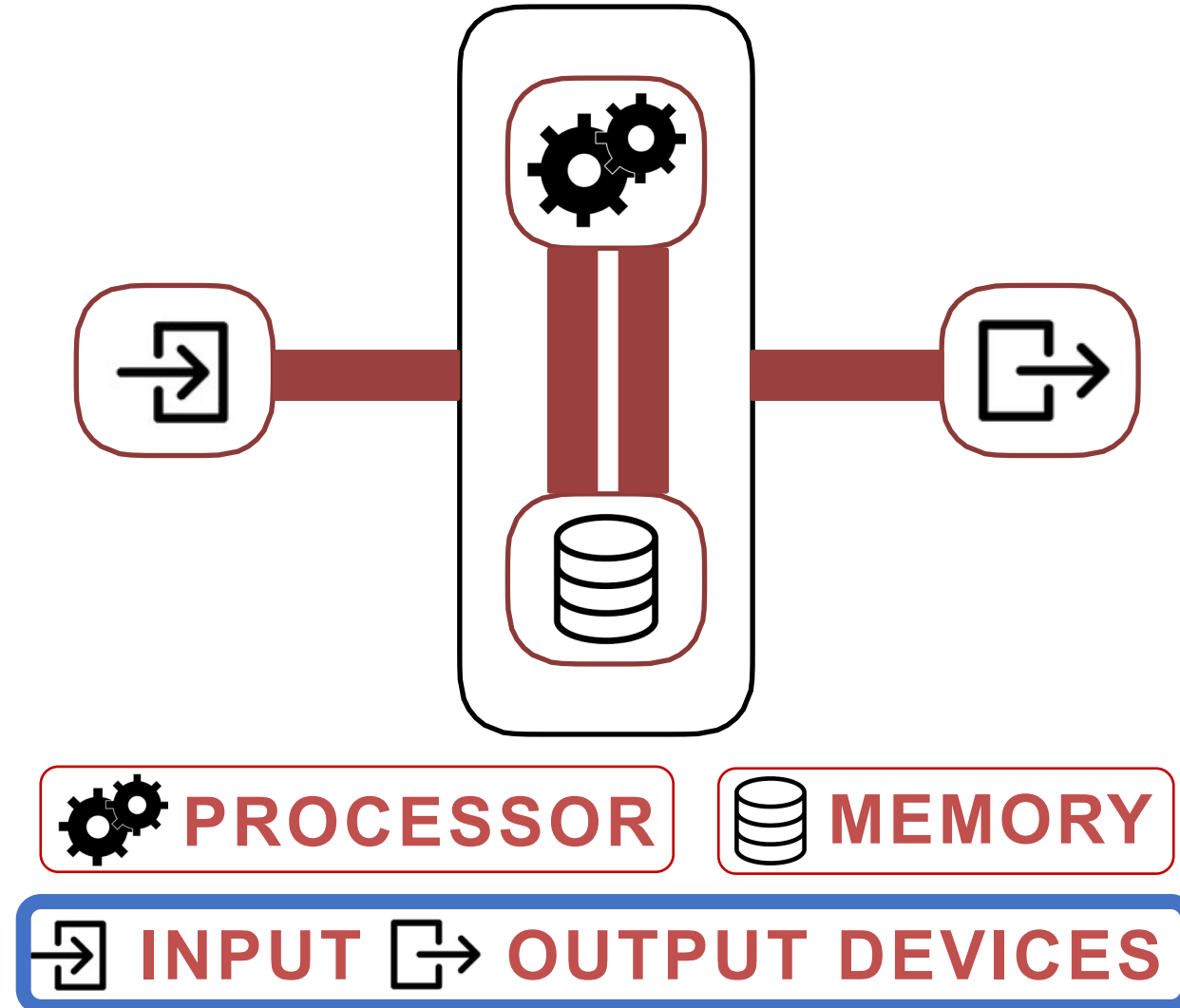
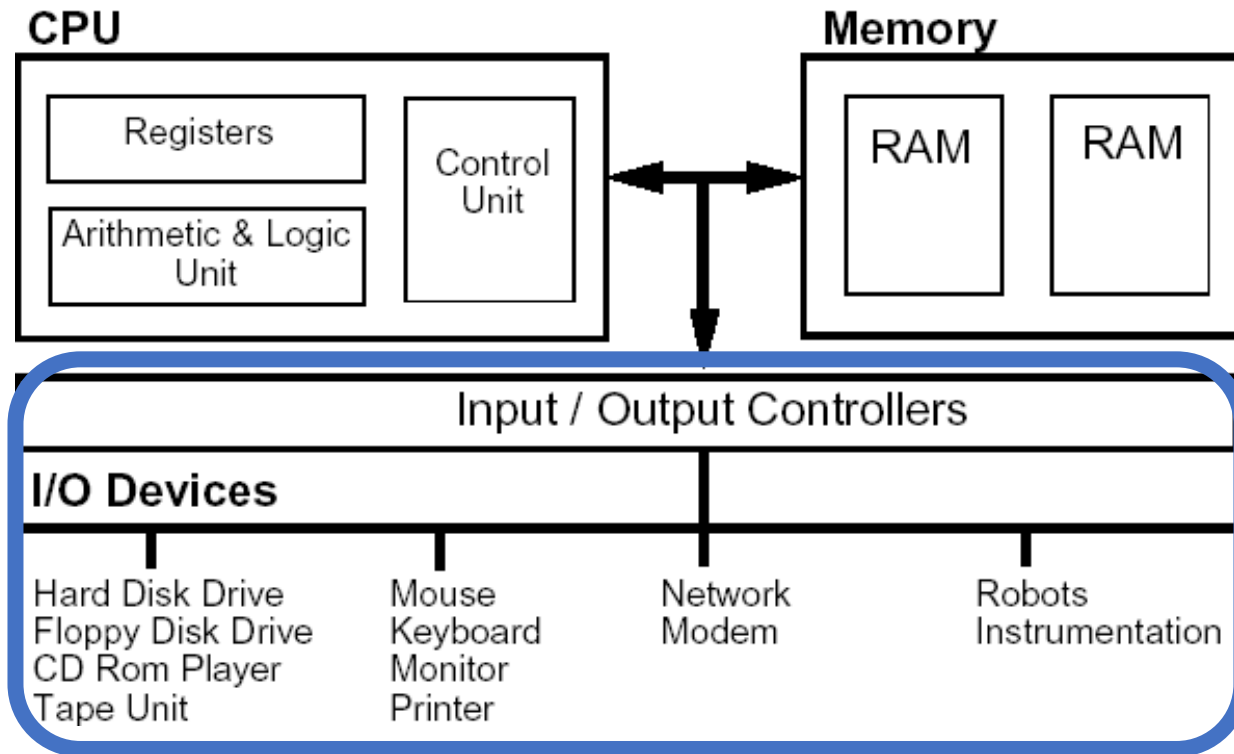
- Interface to the external world

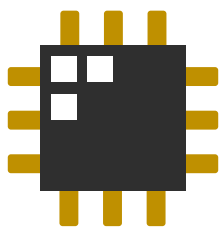




A computer system

- Breaking apart a computer...



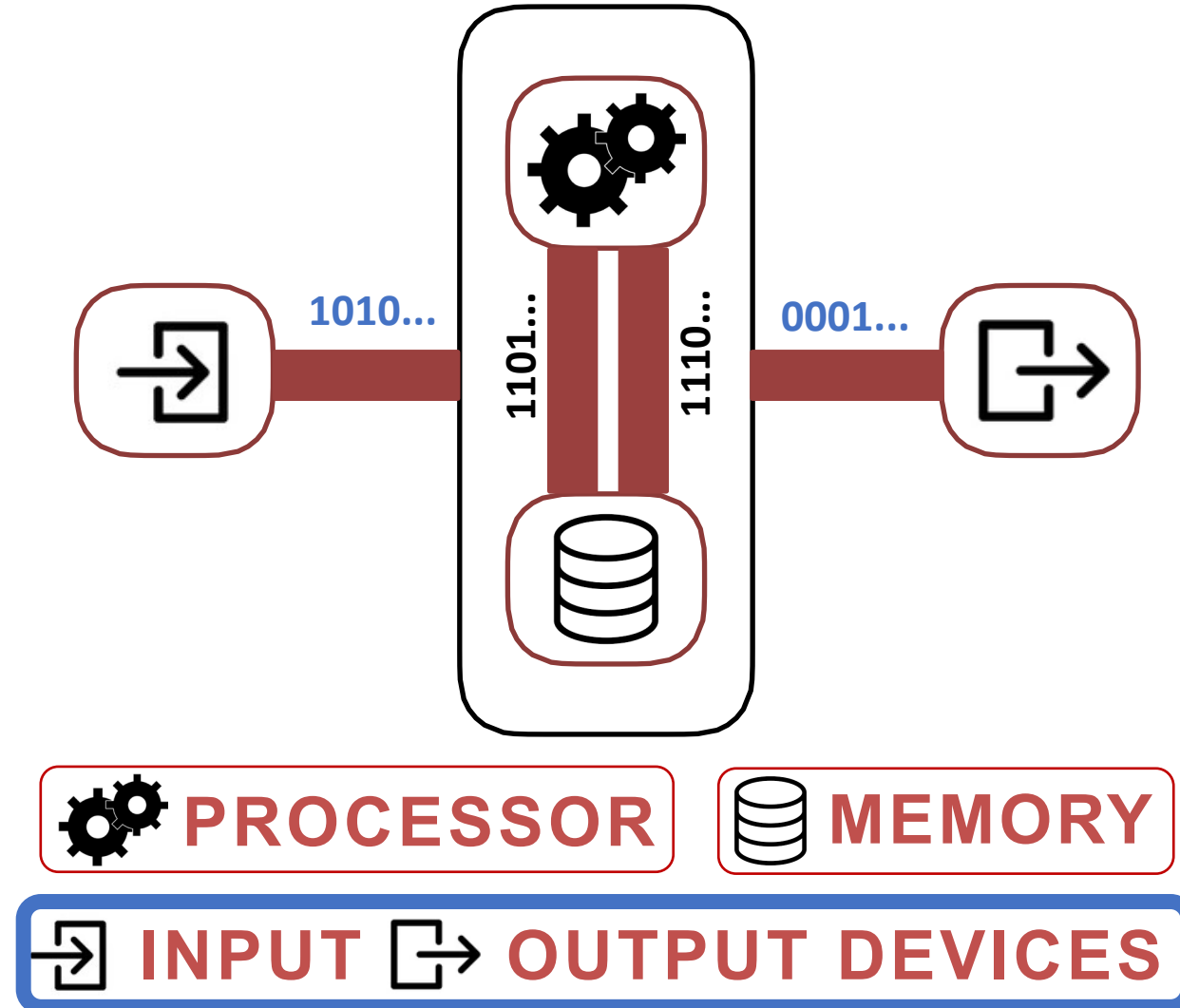
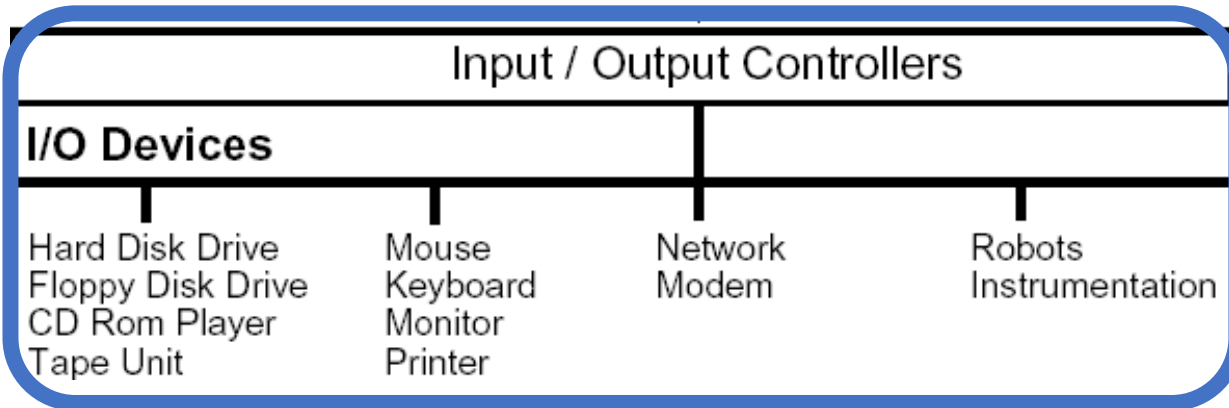


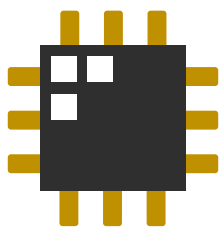
A computer system

- Breaking apart a computer...

If processors can only handle 0s and 1s...

How do we interface with different I/O peripherals?



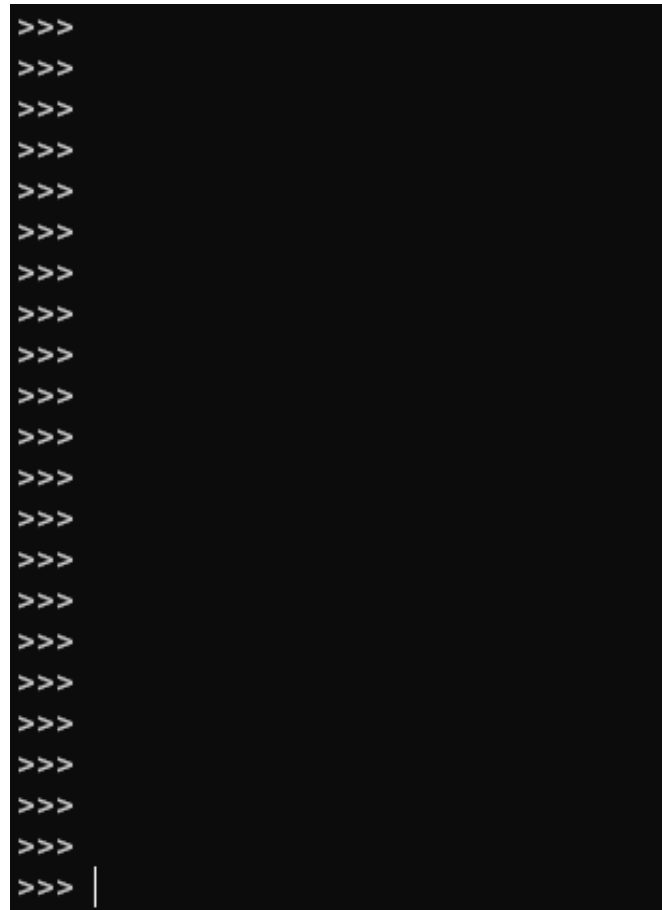


Printing characters

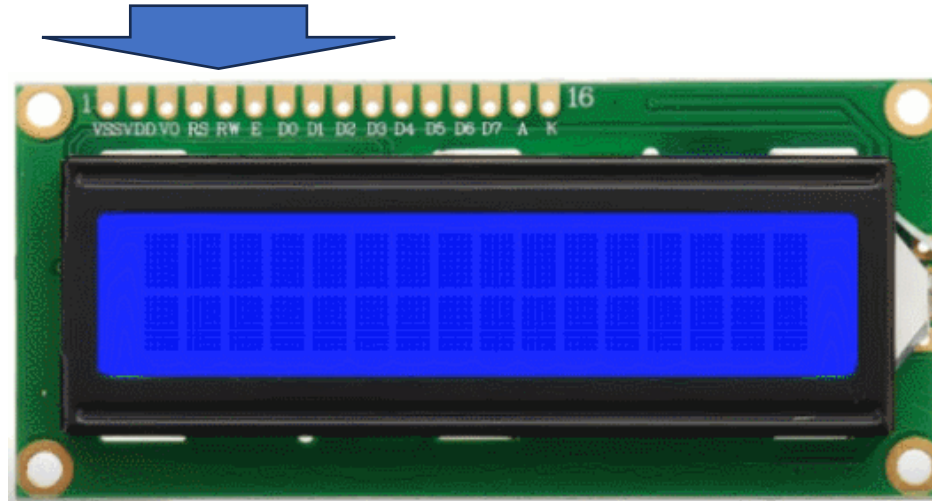
- Characters that you see on your computer screen have hexadecimal equivalents
 - Mapping is defined by the American Standard Code for Information Interchange (**ASCII**)

Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
20	SP	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	\$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

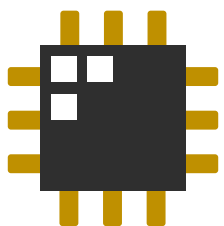
ASCII table



To display characters on interfaces like these, hexadecimal data corresponding to the ASCII code is sent through the data inputs



Characters are drawn from pixels!



How about color information?

- Color information can be displayed by forming a pixel made of red, green, and blue (**RGB**) light-emitting circuit elements
 - Each color can vary in intensity, typically ranging from $[0, 255]$. This is the *range* of 8-bit numbers!

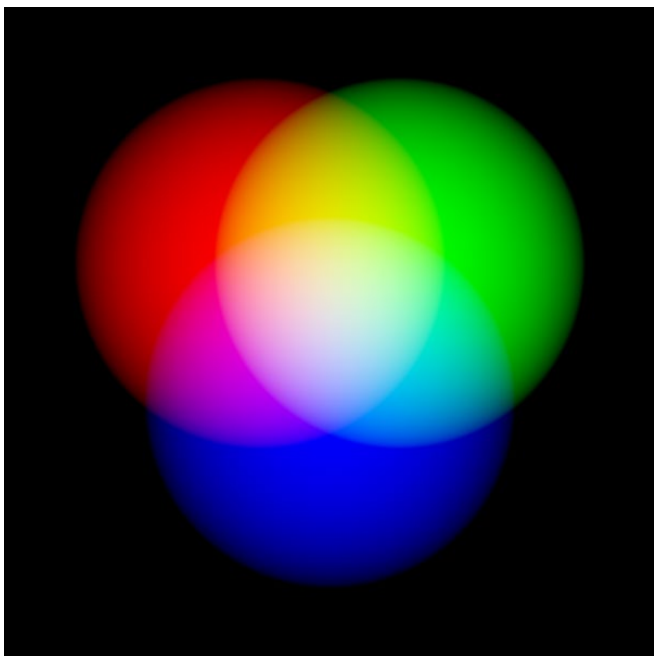
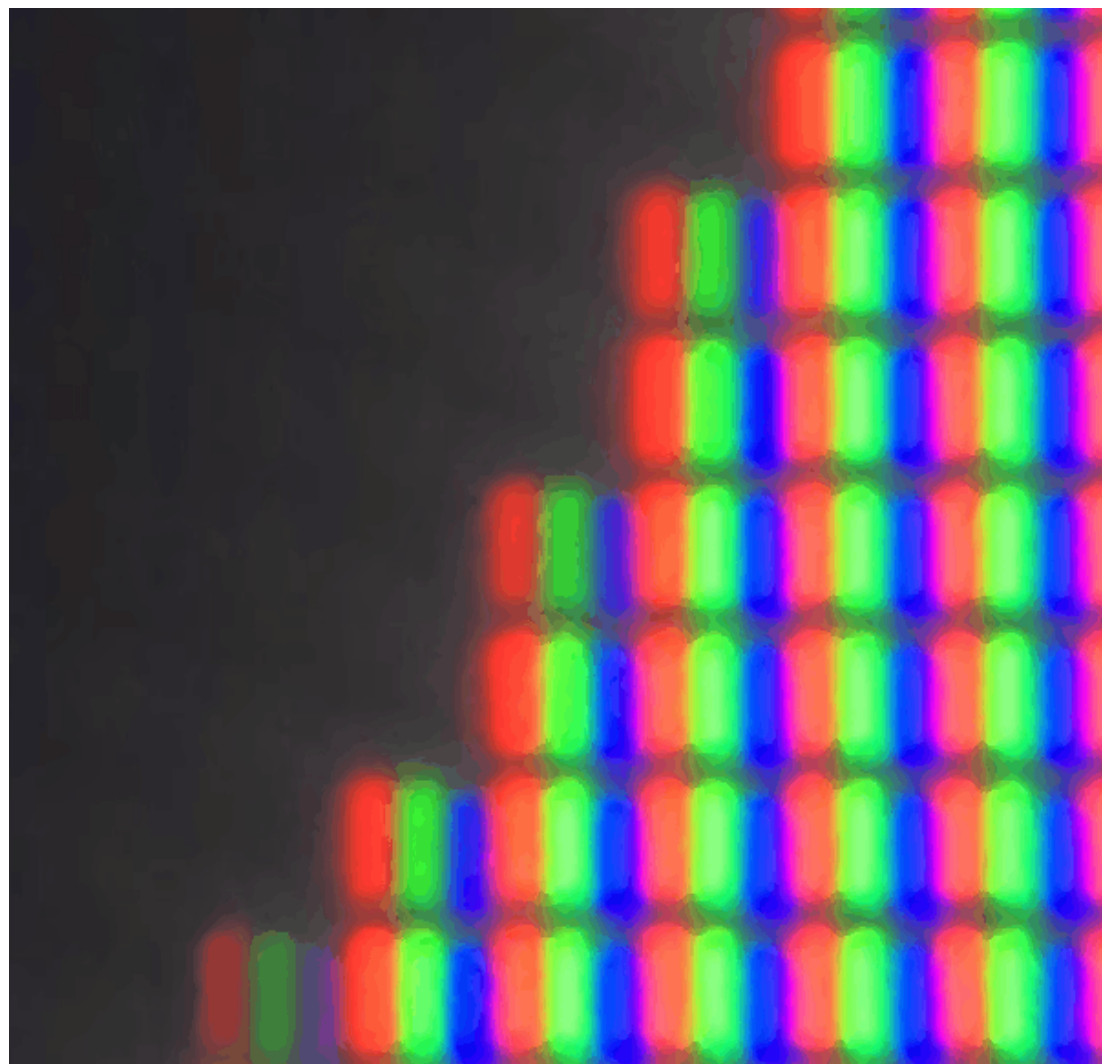
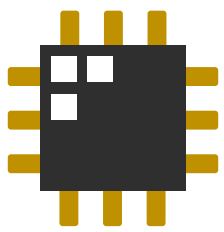


Image credit: commons.wikimedia.org



Video from: <https://www.youtube.com/shorts/T9ryMOrwVZQ>



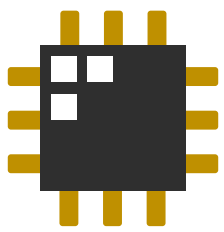
RGB display demo 😊

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	25	20	22	19	22	24	22	83	15	20	80	80	86	70	
2	28	28	28	29	30	32	32	100	26	29	97	100	131	117	
3	19	13	18	18	19	17	21	126	32	44	123	151	198	189	1
4	27	17	29	14	22	20	18	68	14	72	49	104	94	77	
5	30	28	40	24	30	34	27	92	26	96	67	151	150	122	
6	19	11	26	15	19	33	22	102	42	132	105	219	227	181	1
7	18	19	34	12	15	17	44	23	13	66	8	53	35	77	
8	24	30	44	19	28	33	61	33	25	83	28	76	71	105	1
9	12	16	33	12	34	33	71	33	47	103	52	118	119	152	1
10	30	18	46	28	45	87	15	17	84	29	53	23	83	56	
11	38	29	66	44	72	112	30	27	112	61	89	50	108	90	1
12	23	21	77	44	89	143	53	26	160	110	141	61	146	151	2
13	22	23	34	28	38	38	36	52	97	46	82	46	106	106	
14	33	34	54	37	56	50	49	69	137	84	118	64	136	152	1
15	27	26	65	32	42	40	40	79	186	147	168	86	188	227	1
16	13	36	30	63	33	21	32	82	73	76	80	79	60	96	
17	25	40	34	69	39	24	43	96	91	126	122	111	98	152	1
18	13	25	17	57	25	17	39	60	95	199	188	170	161	225	1
19	47	43	25	25	18	71	78	69	55	101	98	97	77	93	
20	58	49	34	33	21	74	86	84	78	150	148	115	99	142	1
21	54	39	29	29	29	65	61	51	84	217	207	151	122	209	2

Ready Accessibility: Good to go

<https://think-maths.co.uk/spreadsheet/>

400%



Graphics processing

- Graphics processing units (GPUs) *offload* video processing away from the processor.
 - **Computing what to display next takes a lot of processing time**
 - Think of GPUs as dedicated processors only meant to process video!

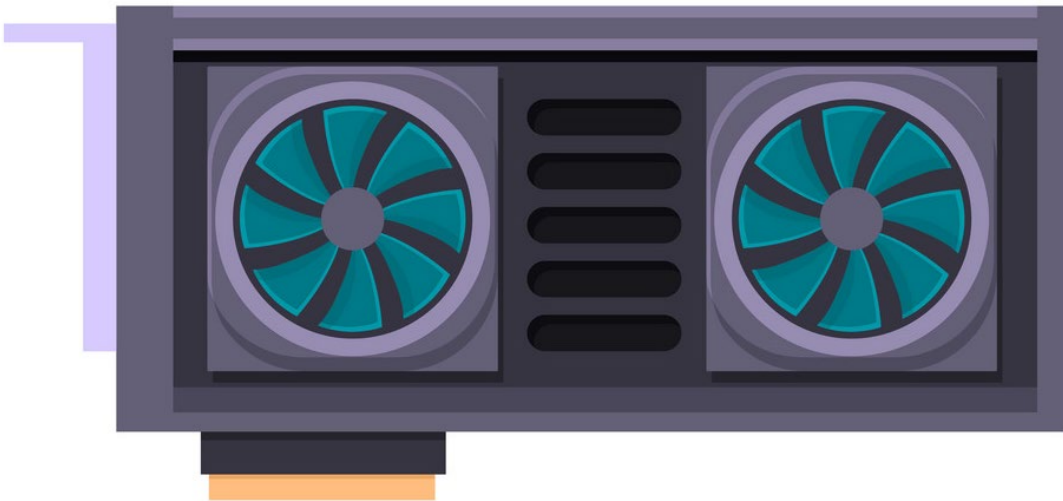
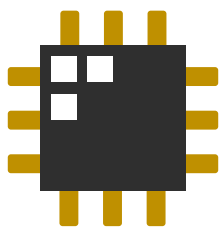


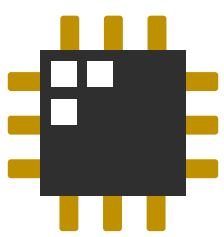
Image credit: vectorstock.com





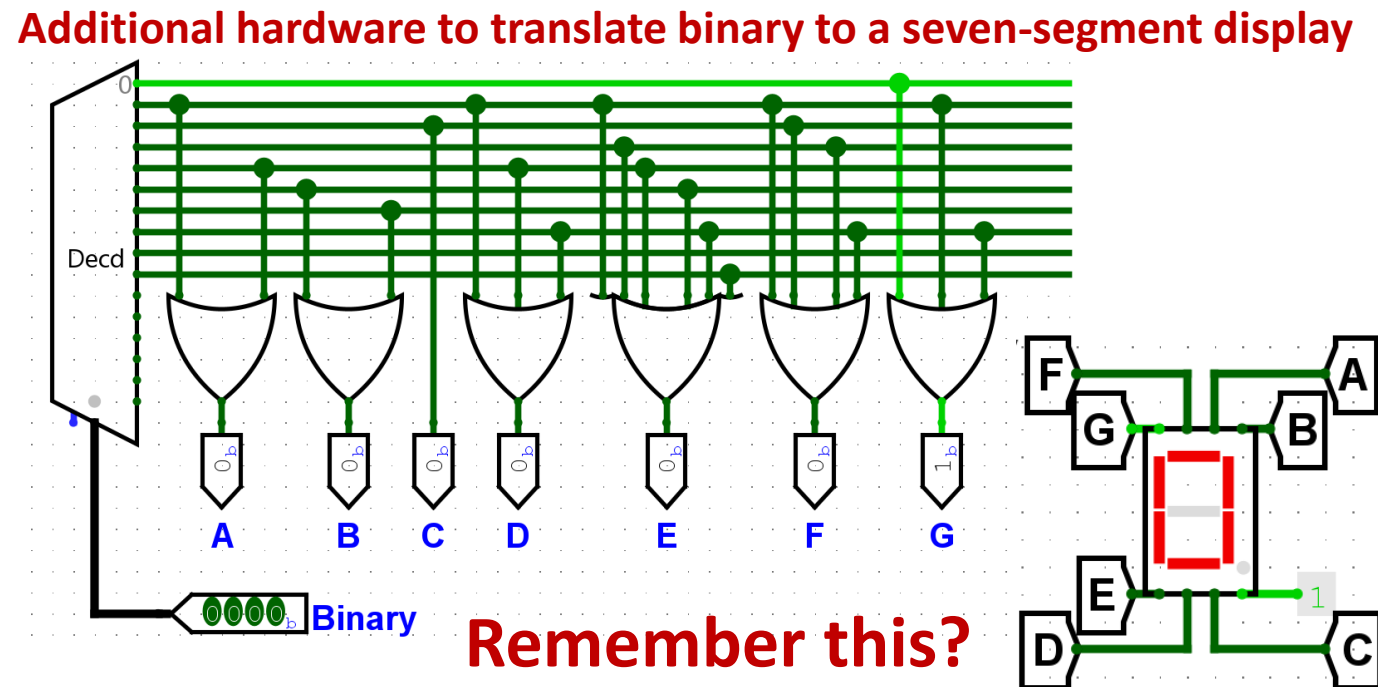
Graphics processing (a *very simple* example)

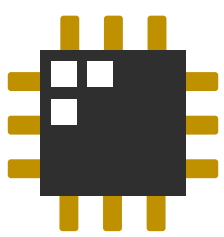
- As much as possible, you want to offload *a lot of processing away* from the processor to dedicated processing units (like GPUs)
 - You want the processor to focus on doing *other* meaningful computations...



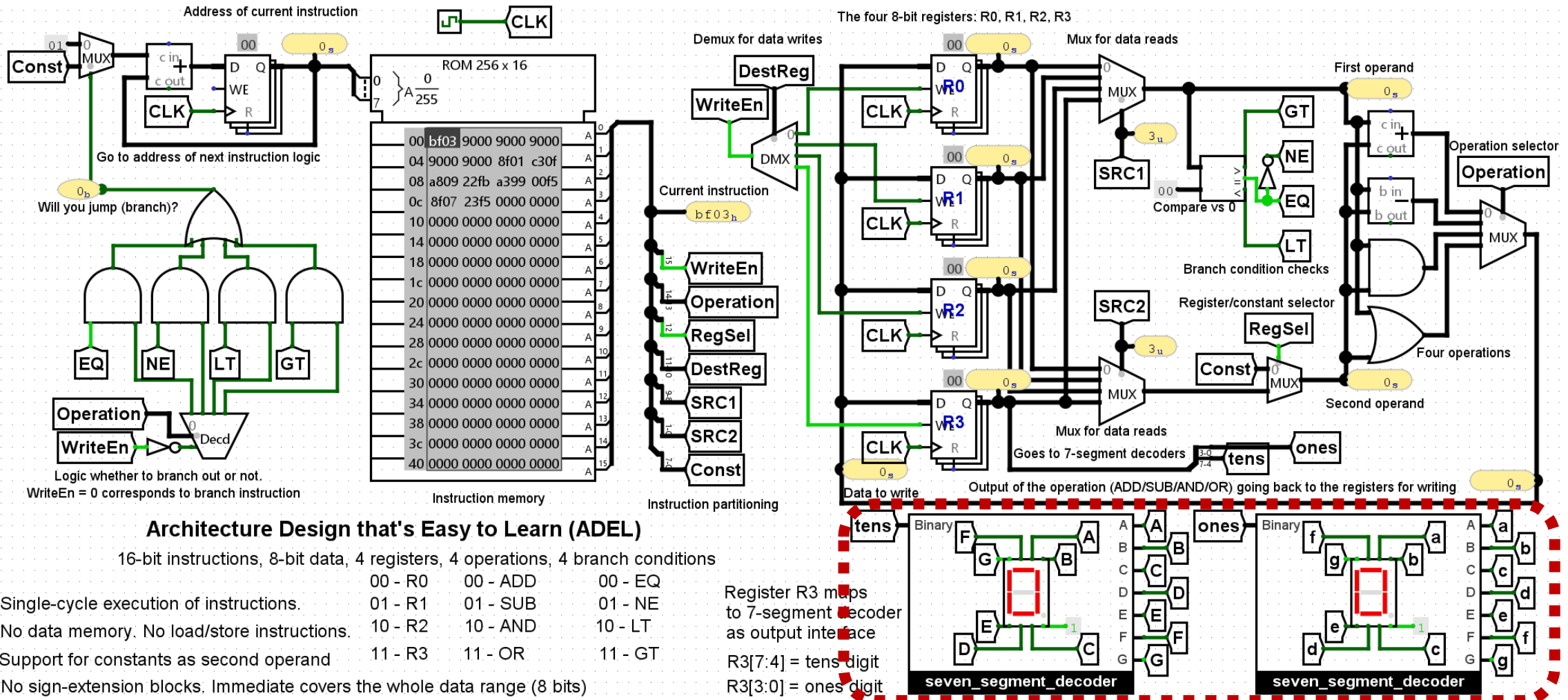
Graphics processing (a *very simple* example)

- As much as possible, you want to offload a lot of processing away from the processor to dedicated processing units (like GPUs)
 - You want the processor to focus on doing *other* meaningful computations...
- For example, if we want our processor to interface with a seven-segment display...
 - We can use another hardware to translate binary code to seven-segment LEDs.
 - The processor only needs to write the binary number to display, the added hardware does the translation.



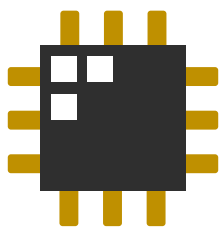


Graphics processing (a *very simple* example)



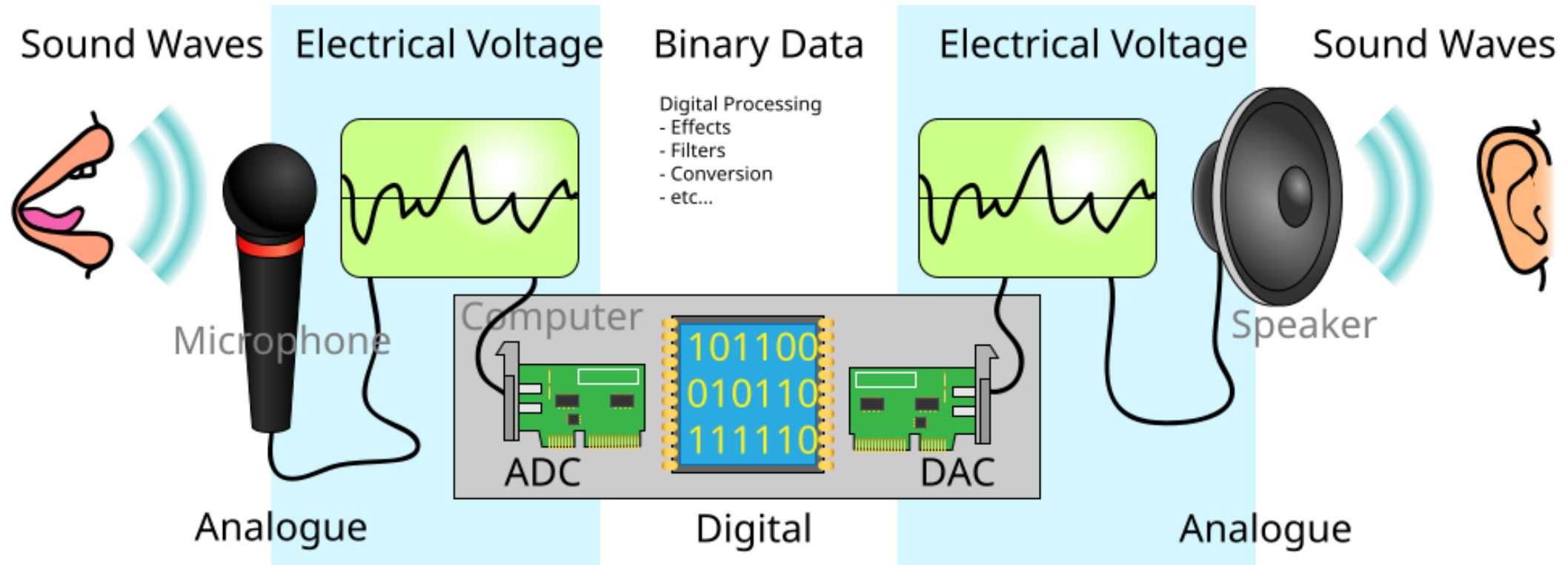
You can probably say that the seven-segment decoders serve as the *very simple* “GPU” for our simple processor!

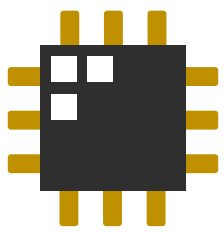
Note: Actual GPUs are more complex than a processor



How about sound information?

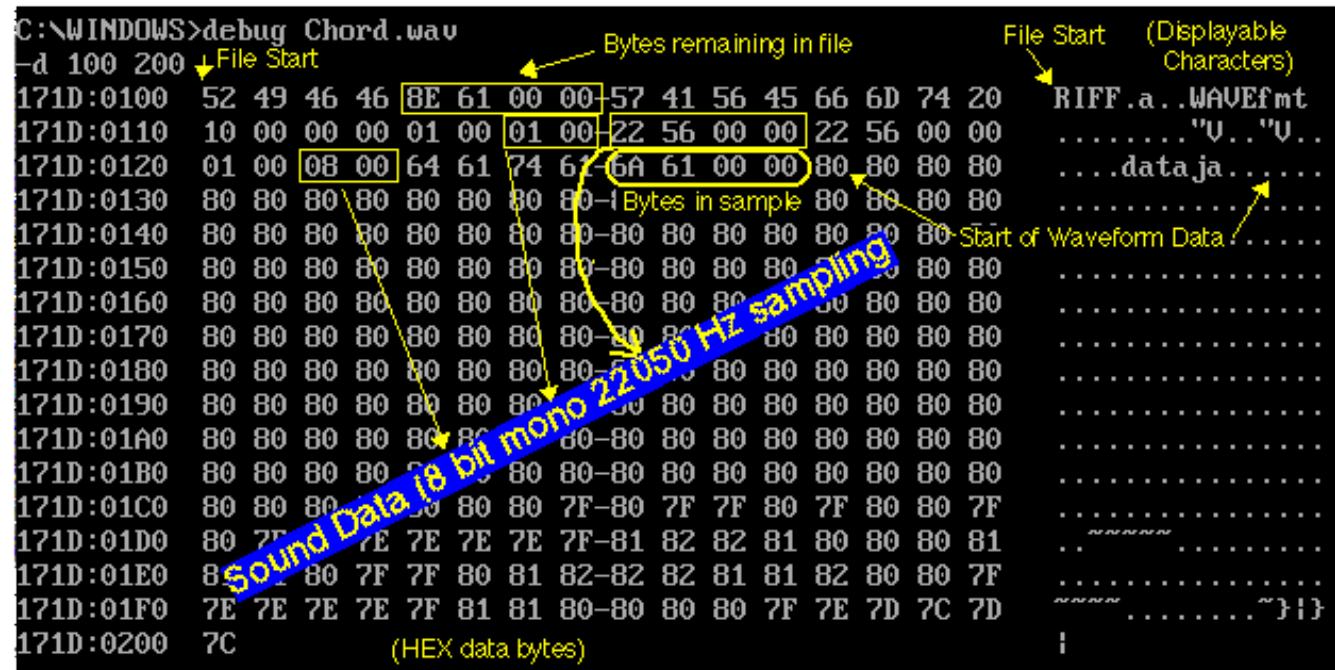
- Sound can be digitized through *analog-to-digital converters* (ADCs) and is converted back to sound waves through *digital-to-analog converters* (DACs)





How about sound information?

- In a computer, sound files are just binary representing the waveform!
 - Some file formats are compressed versions, but they're still waveform data in the end.



**WAVE (.wav) data format
including the file headers**

```
import matplotlib.pyplot as plt
import numpy as np
import wave
import sys

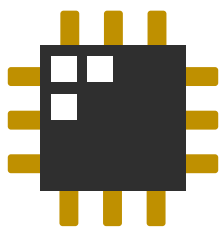
spf = wave.open("test.wav", "r")

# Extract Raw Audio from Wav File
signal = spf.readframes(-1)
signal = np.fromstring(signal, "Int16")
fs = spf.getframerate()

Time = np.linspace(0, len(signal) / fs, num=len(signal))

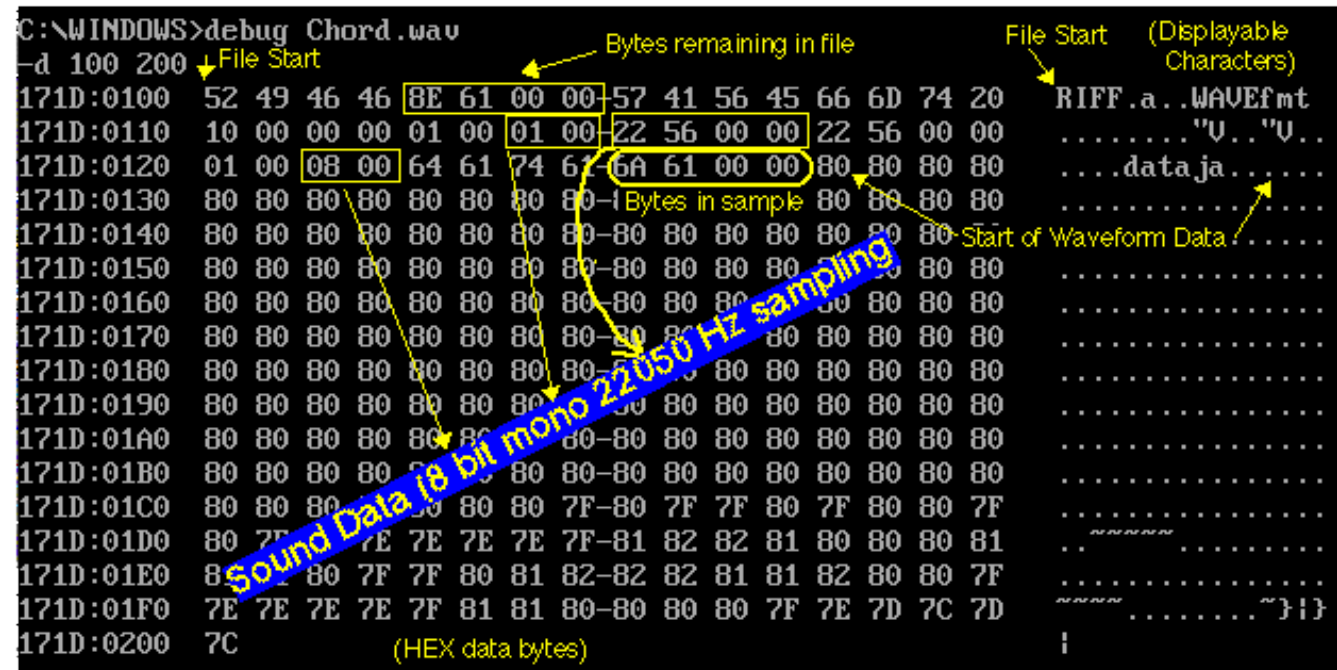
plt.figure(1)
plt.title("Waveform")
plt.plot(Time, signal)
plt.show()
```

**Simple python code to display
the (.wav) waveform**

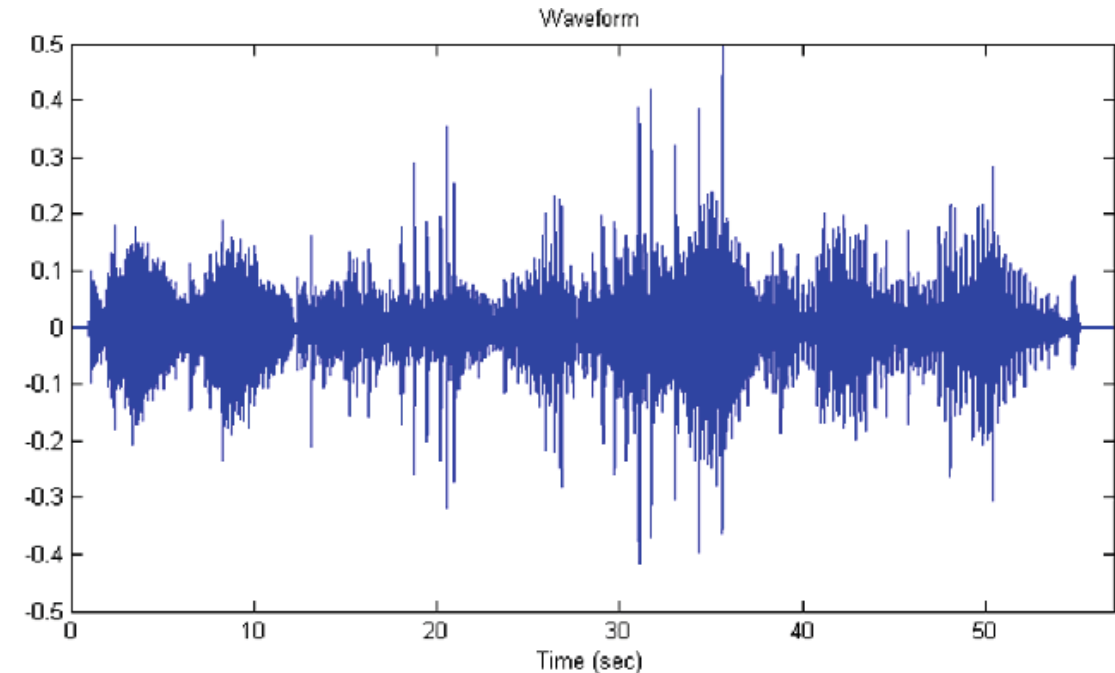


How about sound information?

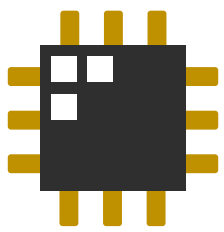
- In a computer, sound files are just binary representing the waveform!
 - Some file formats are compressed versions, but they're still waveform data in the end.



**WAVE (.wav) data format
including the file headers**

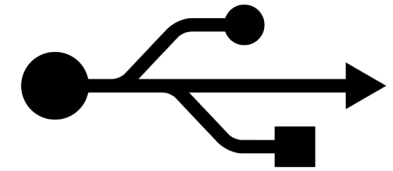


**Plotting the .wav file after
removing the file headers**



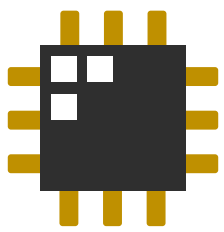
Communication protocols

- Peripherals connected to the computer would communicate in binary following a specific *protocol* (set of rules to govern what a transaction should look like)
 - Example: Universal Serial Bus (USB) protocol



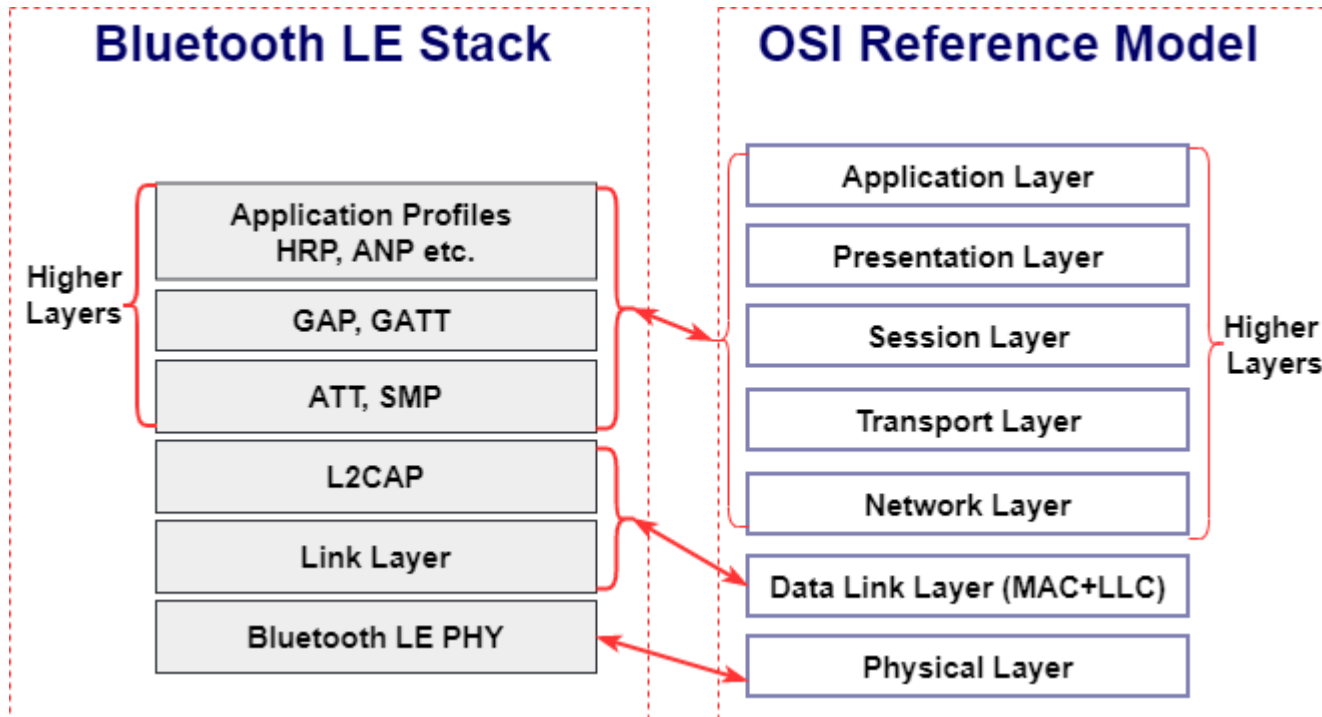
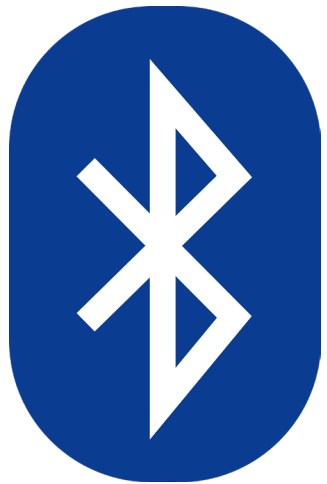
See how the peripheral is just sending a waveform that is just 1s and 0s

The computer would *parse* the waveform and interpret the data

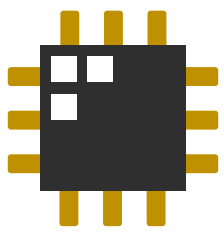


Communication protocols

- Peripherals connected to the computer would communicate in binary following a specific *protocol* (set of rules to govern what a transaction should look like)
 - Bluetooth and WiFi also work following a specific protocol stack
 - Once the data is properly decoded at the receiver, everything is just binary!



No need to know what the different terms are. This is just for appreciation. 😊



Communication protocols

- Once the data is properly decoded at the receiver, everything is just binary!

Capturing from Wi-Fi: en0 — KaliLinux-2 Ethernet0 to Ethernetswitch-1 Ethernet1

http

No.	Time	Source	Destination	Protocol	Length	Info
...	6.058...	10.158.34.54	128.119.245.12	HT...	5...	GET /wireshark-labs/HTTP-wire...
...	6.330...	128.119.245.12	10.158.34.54	HT...	5...	HTTP/1.1 200 OK (text/html)

Frame 109: 548 bytes on wire (4384 bits), 548 bytes captured (4384 bits) on interface

Ethernet II, Src: Apple_cb:f0:98 (08:6d:41:cb:f0:98), Dst: HewlettP_c7:c2:c0

Internet Protocol Version 4, Src: 10.158.34.54, Dst: 128.119.245.12

Transmission Control Protocol, Src Port: 57248, Dst Port: 80, Seq: 1, Ack: 1,

Hypertext Transfer Protocol

GET /wireshark-labs/HTTP-wire... HTTP/1.1\r\n

Host: gaia.cs.umass.edu\r\n

Connection: keep-alive\r\n

DNT: 1\r\n

Upgrade-Insecure-Requests: 1\r\n

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537...

0000 64 51 06 c7 c2 c0 08 6d 41 cb f0 98 08 00 45 00 dQ.....m A.....E.

0010 02 16 00 00 40 00 40 06 96 8a 0a 9e 22 36 80 77@.@."6.w

0020 f5 0c df a0 00 50 3b 7b d5 c4 9c b2 29 a8 80 18P;{)

0030 08 0a 47 16 00 00 01 01 08 0a 1d 64 94 64 ad 26 ..G..... ..d.d.&

0040 bc 5d 47 45 54 20 2f 77 69 72 65 73 68 61 72 6b .]GET /w ireshark

0050 2d 6c 61 62 73 2f 48 54 54 50 2d 77 69 72 65 73 -labs/HT TP-wires

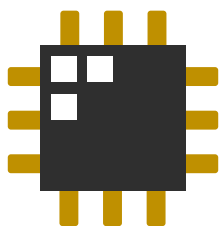
0060 68 61 72 6b 2d 66 69 6c 65 31 2e 68 74 6d 6c 20 hark-fil e1.html

0070 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 HTTP/1.1 ..Host:

- Shout out to the Communication section of EEE 113!

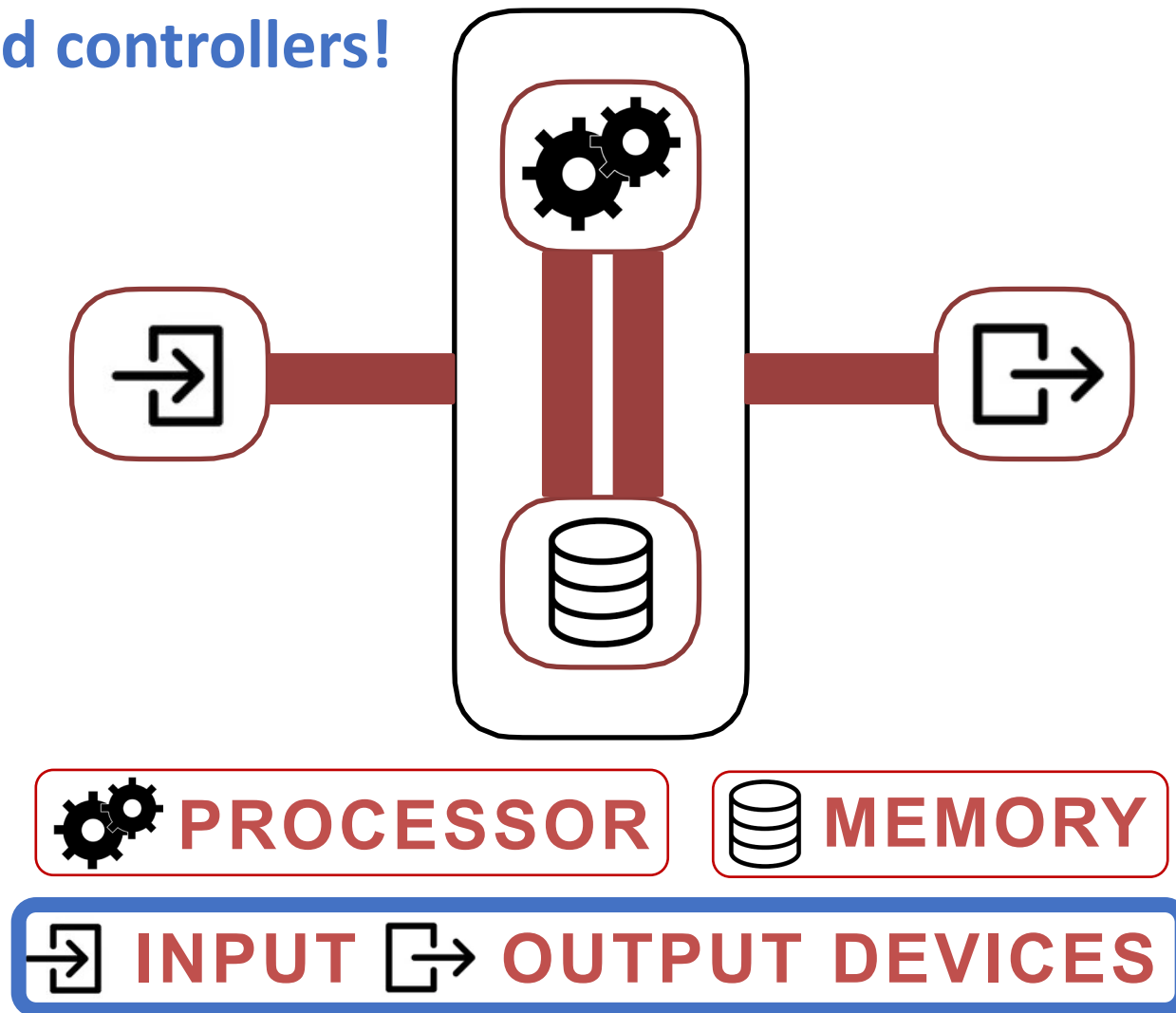
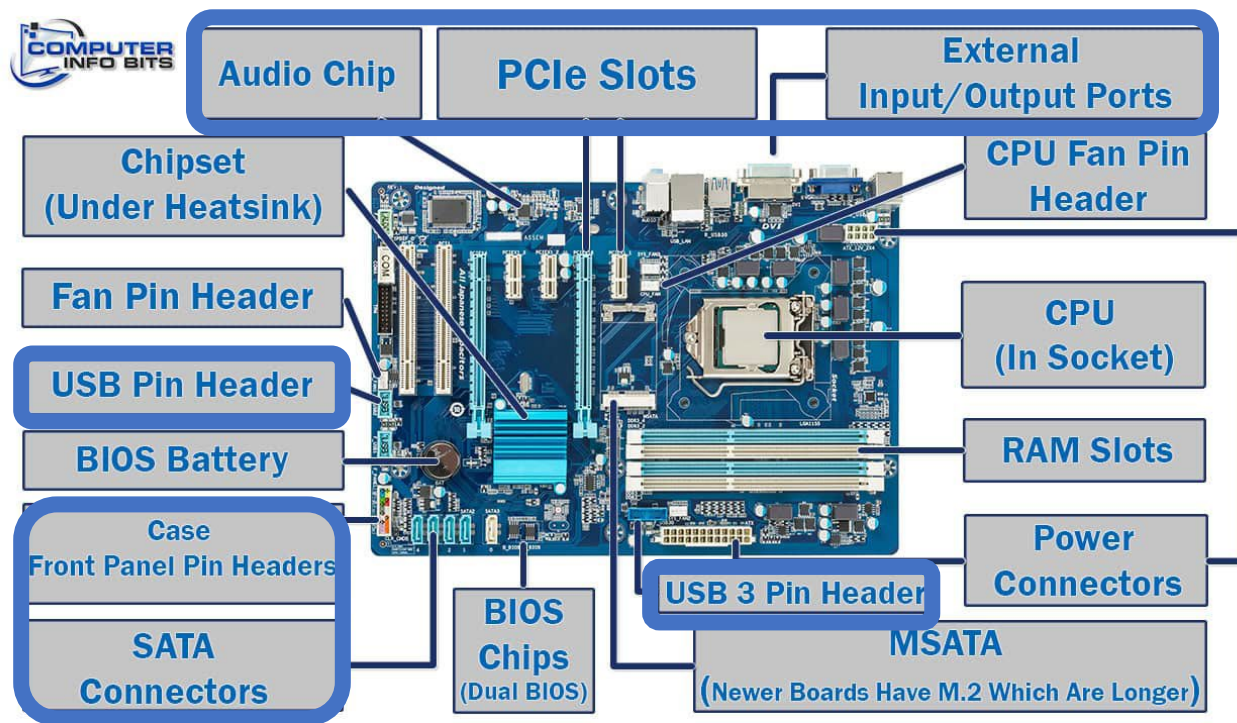
Wireshark capture of a website access

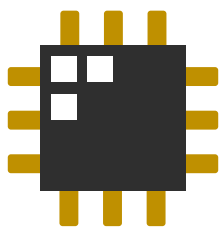
Note how everything is just binary (hex) and is just translated into ASCII text



A computer system

- Different I/O devices, different dedicated controllers!





The unifying example for EEE 113

- **Smart I-Lab Project (Rom 308)**

EEE's vision of the
Future of Engineering Instruction

→ Smart i-Lab

A green,

future-ready,

and smart

instructional laboratory



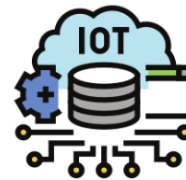
Green Energy

Powered by renewable
energy



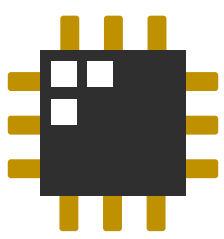
Collaborative and
Hybrid Learning

Equipped with
teleconferencing and
collaborative learning setups



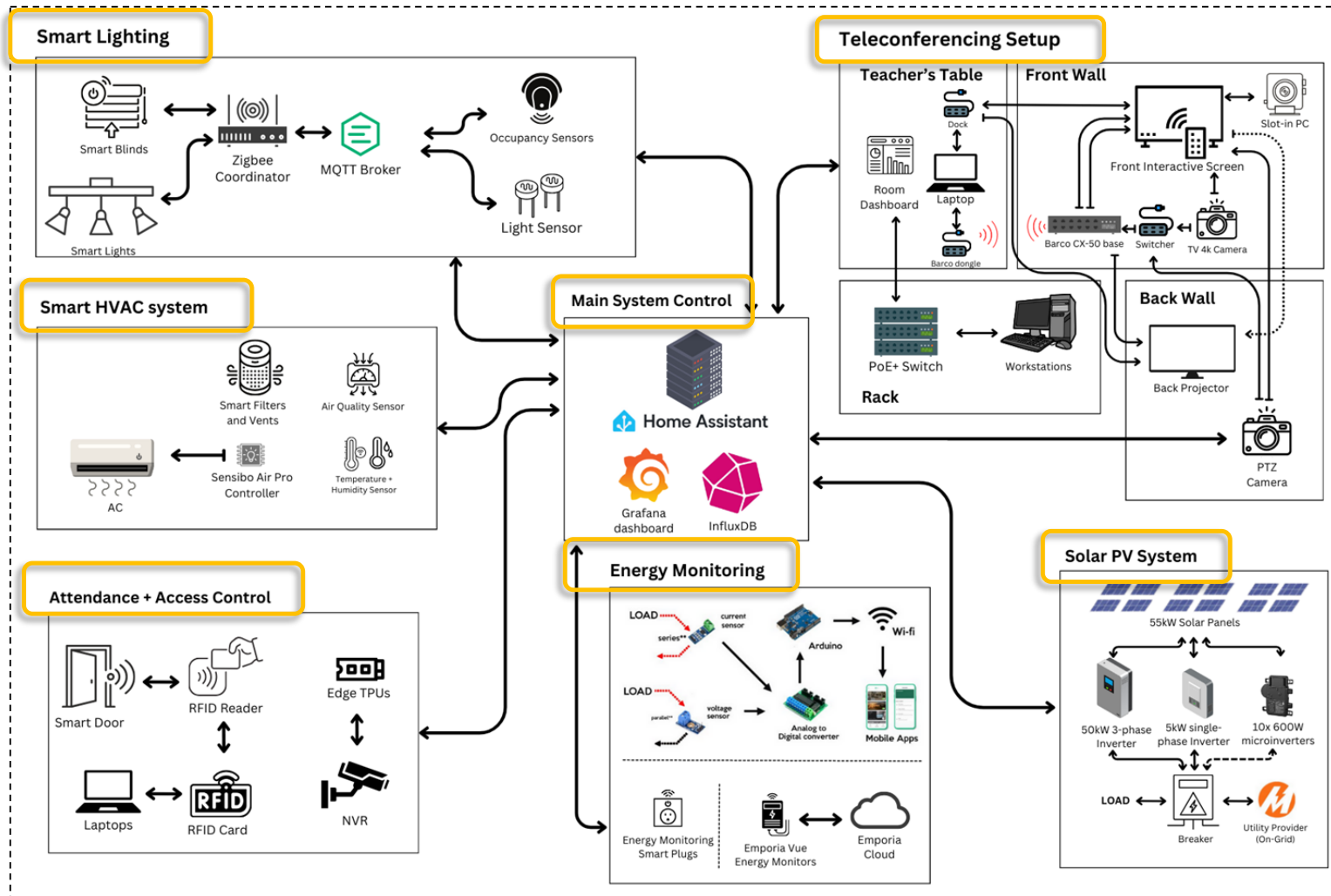
IoT-Driven

Embedded with sensing and
automation features for
environmental and energy
monitoring, and smart control

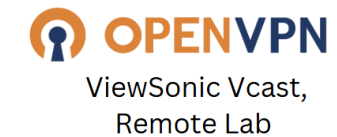


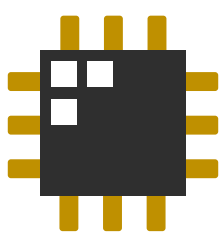
The unifying example for EEE 113

- Smart I-Lab Project (Rom 308)



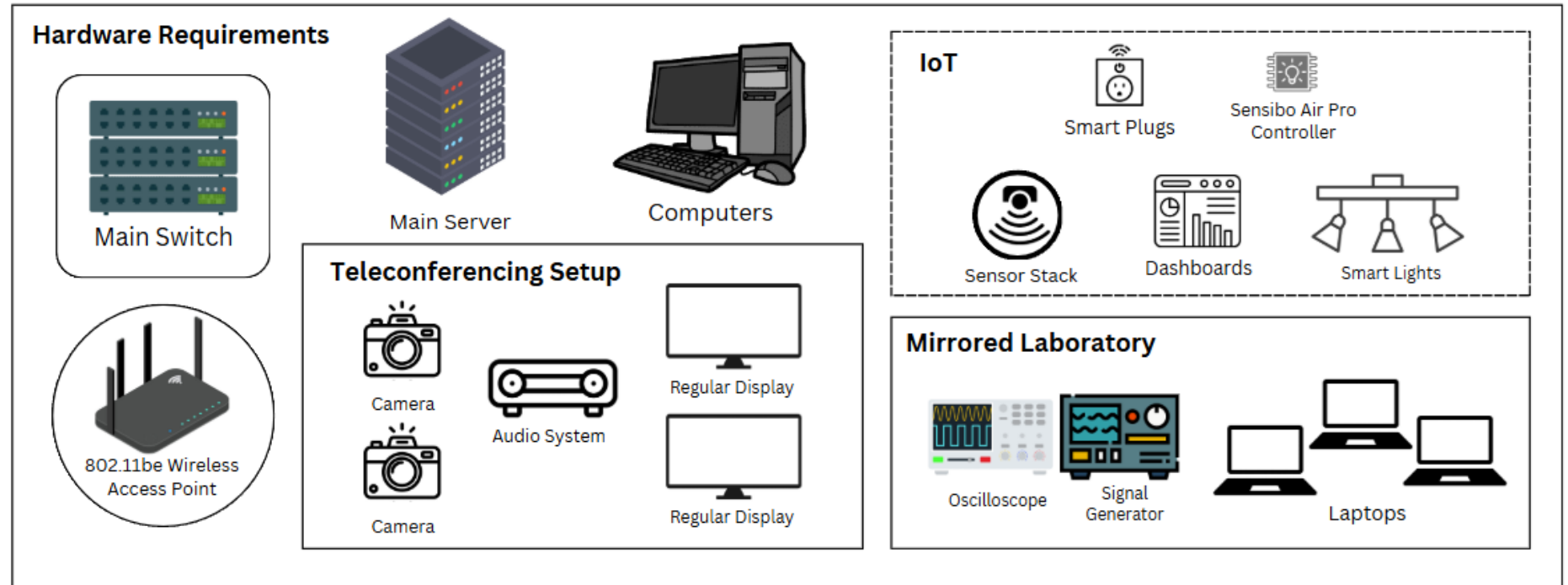
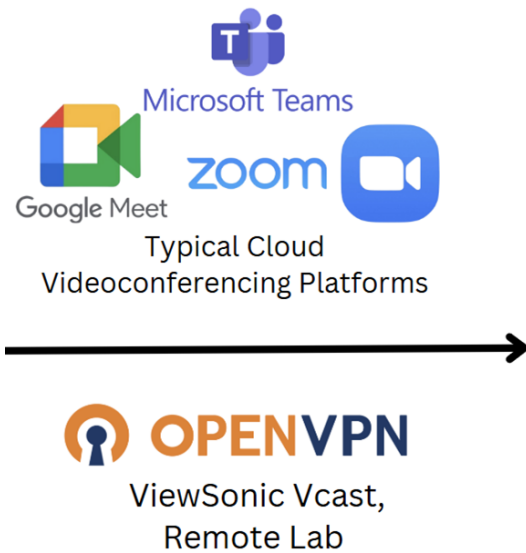
Typical Cloud
Videoconferencing Platforms

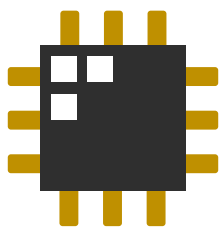




The unifying example for EEE 113

- **Smart I-Lab Project (Rom 308)**

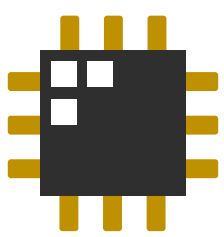




Going back...

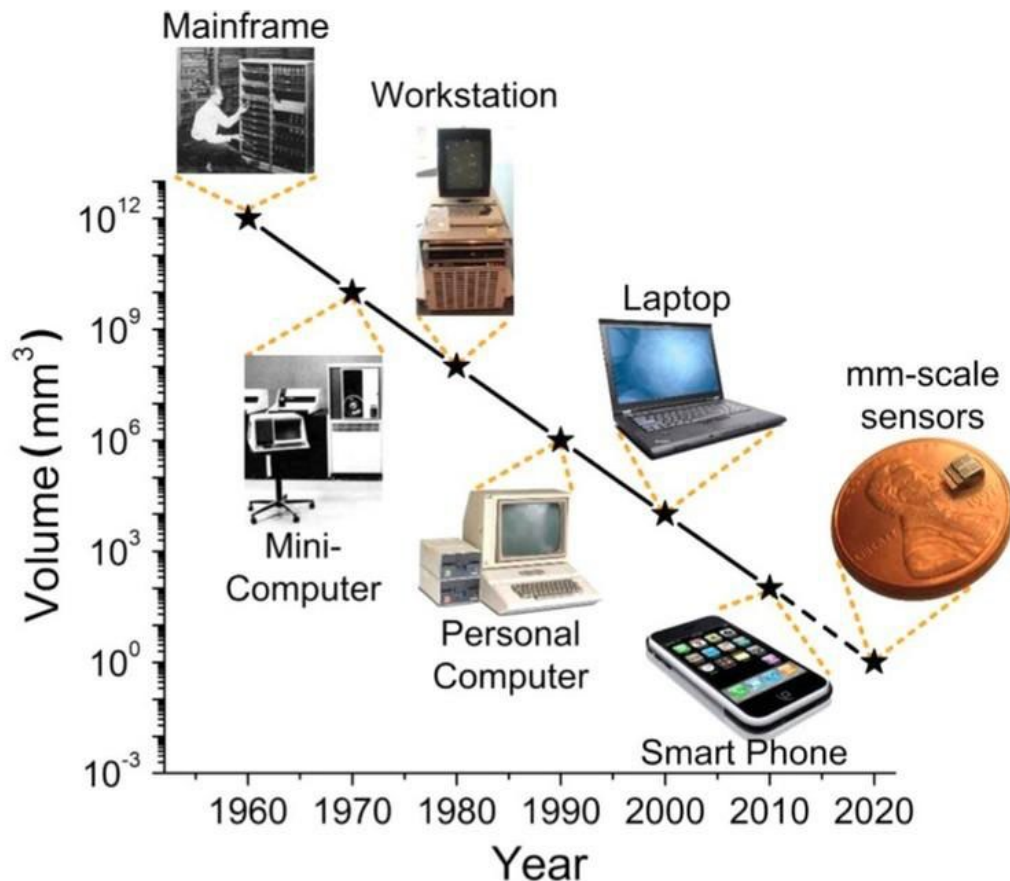
What are the current design trends in computing?

What's next?

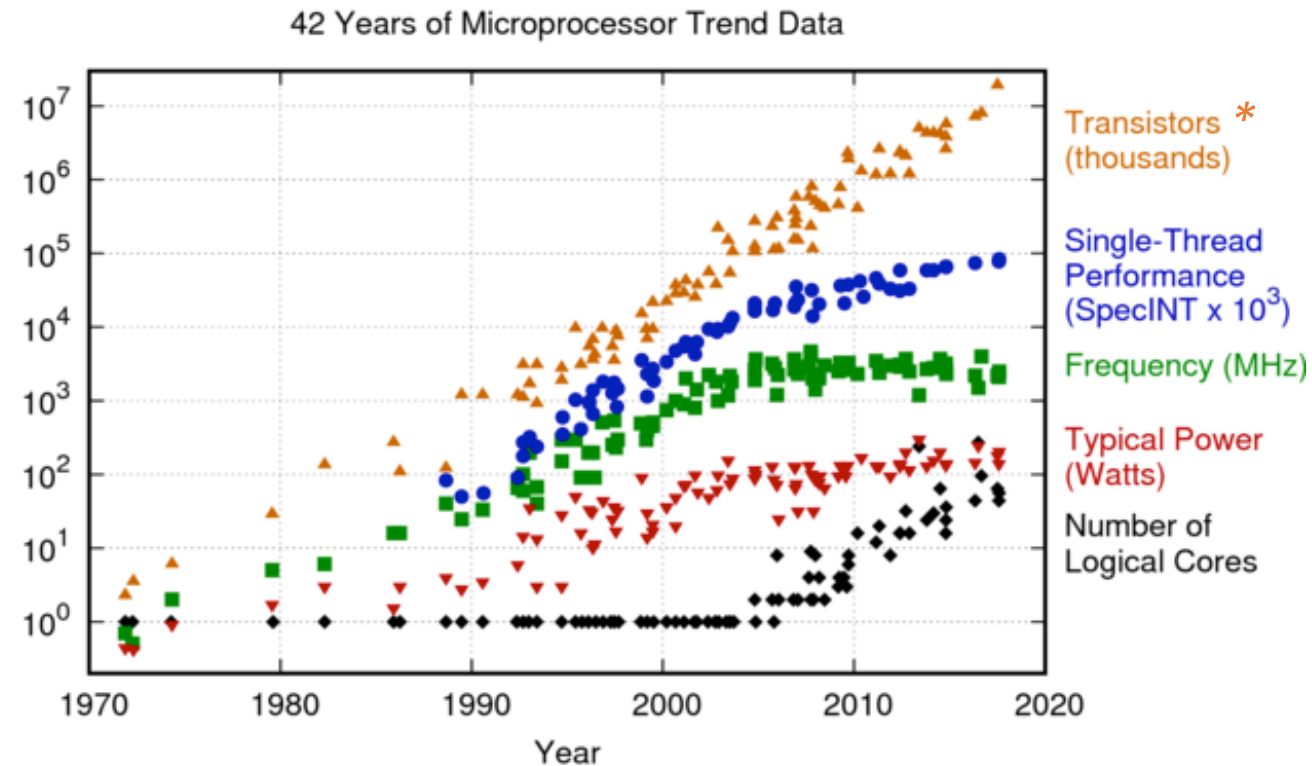


Processing design trends

The size of a computer becomes smaller...



The complexity of a computer increases...

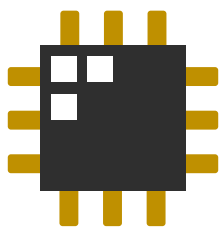


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

6.9 billion

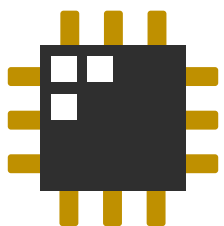
Transistors





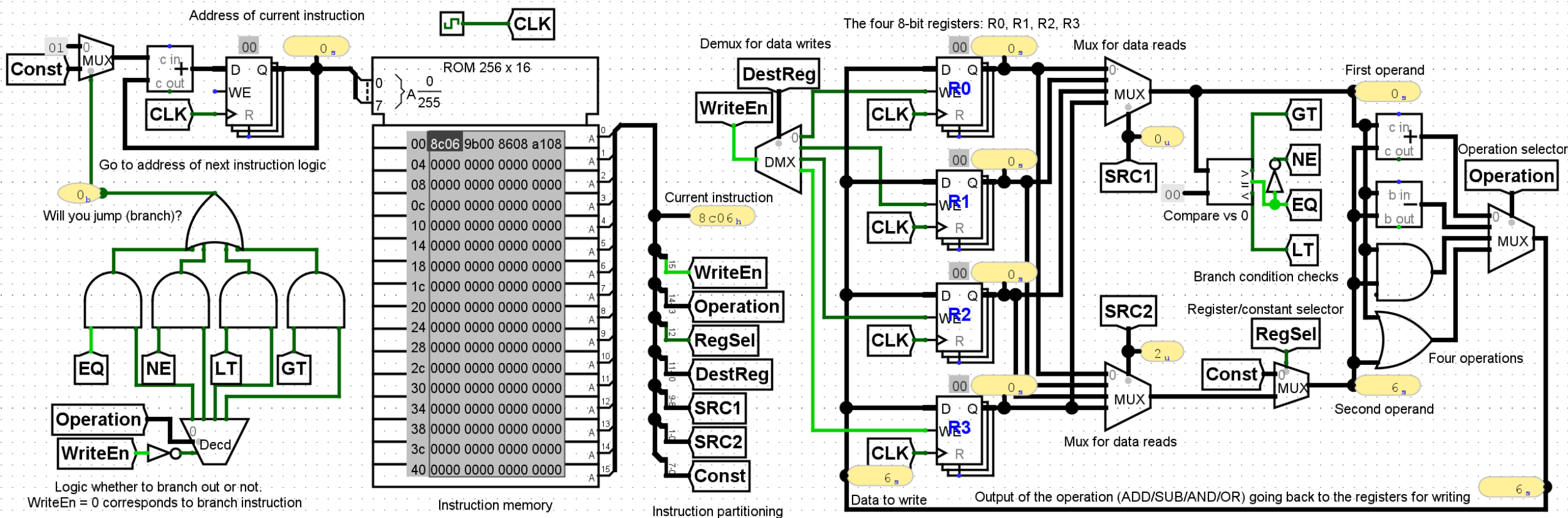
What are transistors anyway?

- To answer this, let's go all the way back...

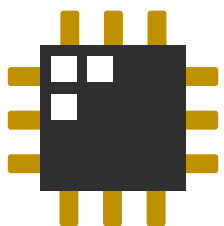


What are transistors anyway?

- To answer this, let's go all the way back...

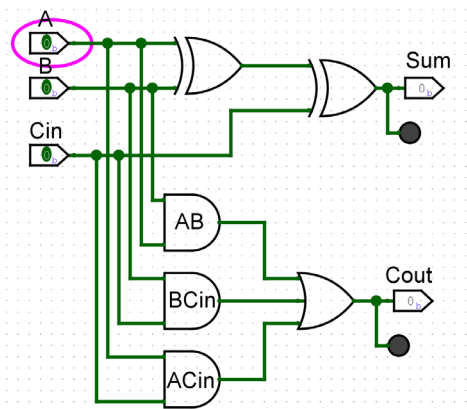


Every component of this processor is made out of logic gates



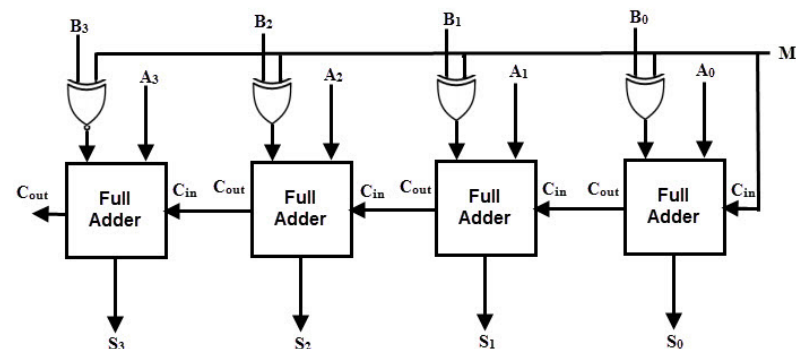
Going all the way back...

Binary arithmetic: adders/subtractors

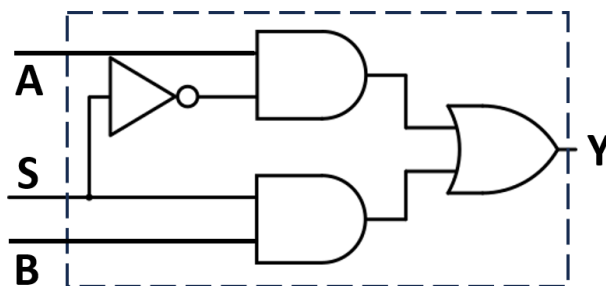


$$S = A \oplus B \oplus C_{in}$$

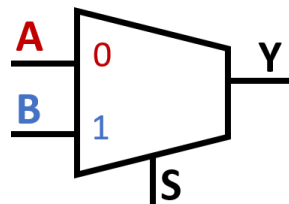
$$C_{out} = BC_{in} + AC_{in} + AB$$



Logic blocks: mux/decoders

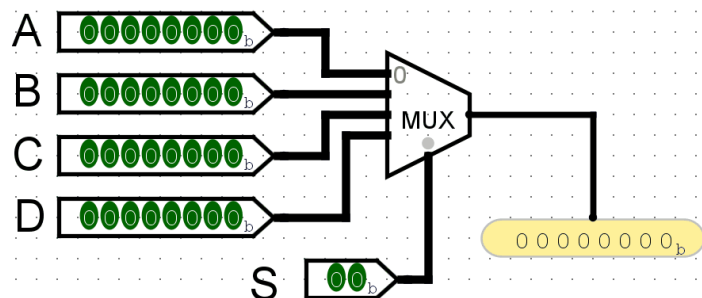


$$Y = S'A + SB$$

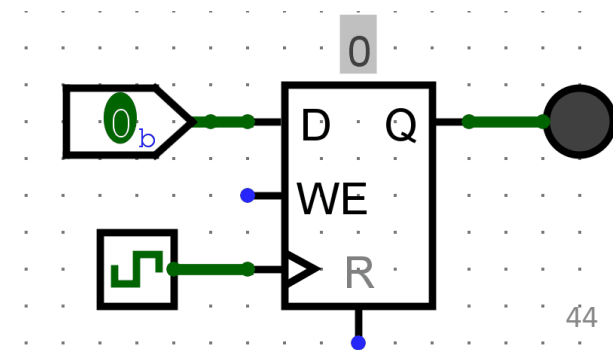
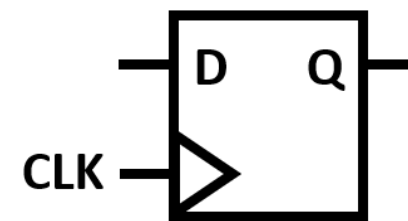
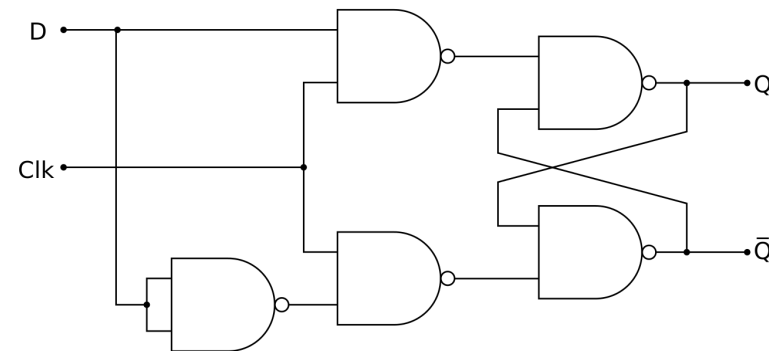


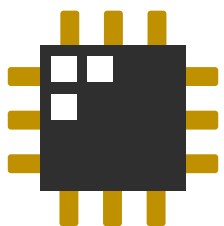
If $S = 0$, $Y = A$

If $S = 1$, $Y = B$



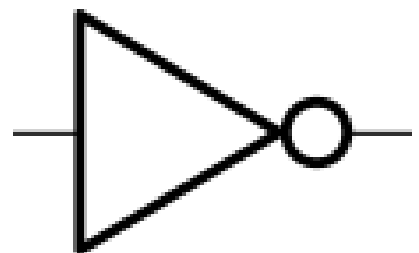
Memory: flipflops/registers



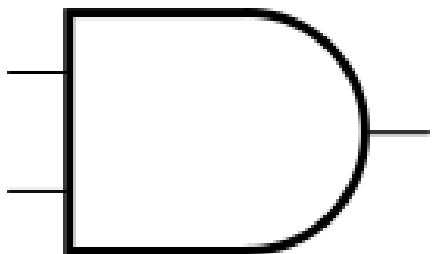


Going all the way back...

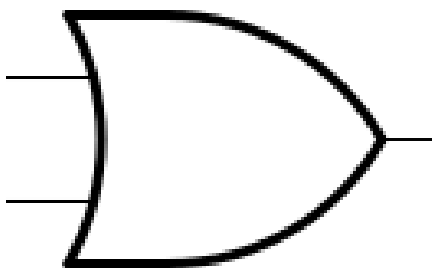
We can think of logic gates as networks of switches



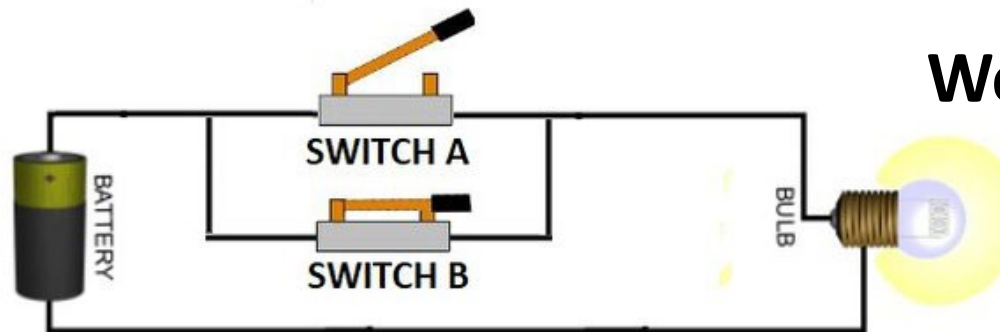
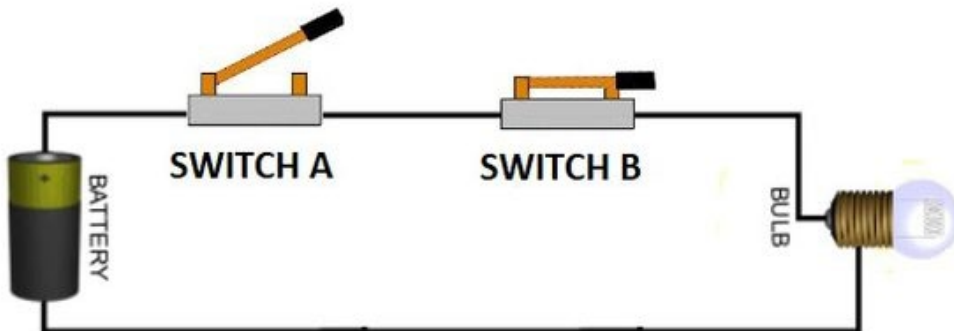
NOT gate



AND gate

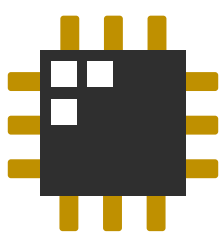


OR gate



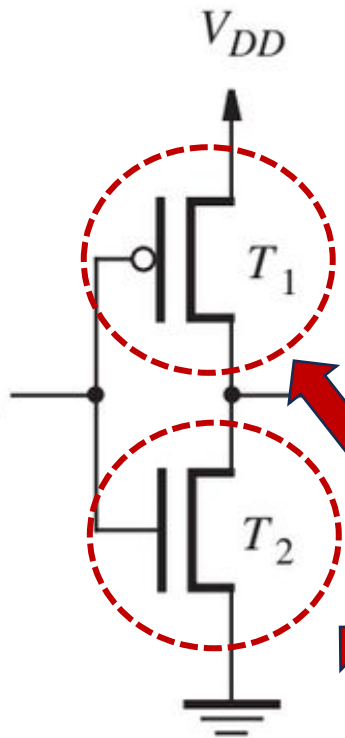
Week 1

Application Software		Programs	higher level
Operating Systems		Device Drivers	
Architecture		Instructions Registers	
Micro-architecture		Datapaths Controllers	
Logic		Adders Memories	
Digital Circuits		AND Gates NOT Gates	
Analog Circuits		Amplifiers Filters	
Devices		Transistors Diodes	
Physics		Electrons	lower level

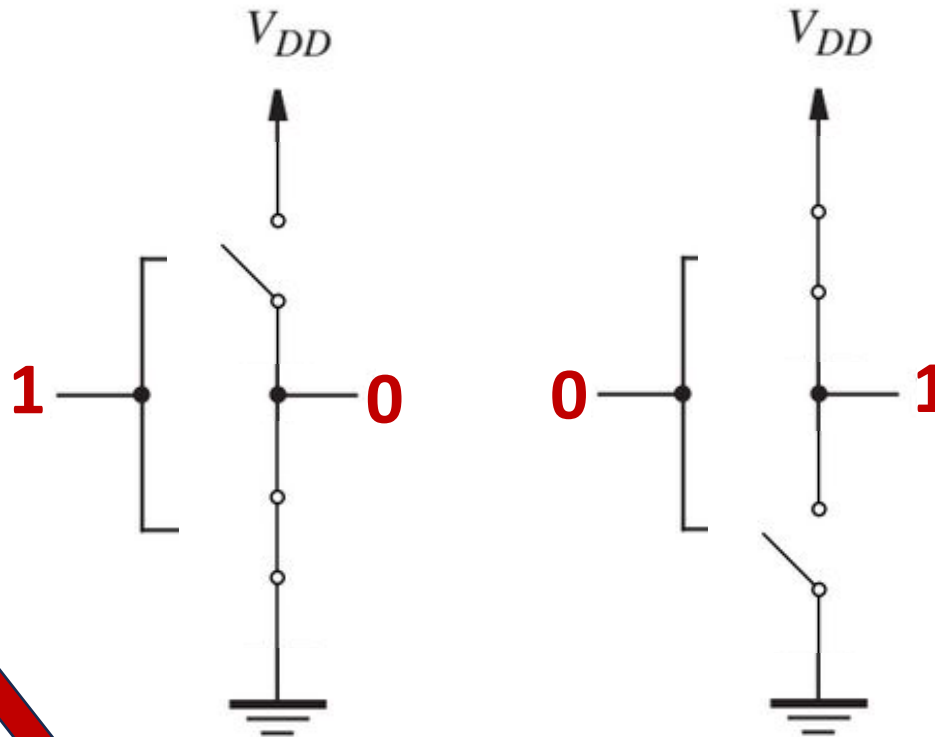


What are transistors anyway?

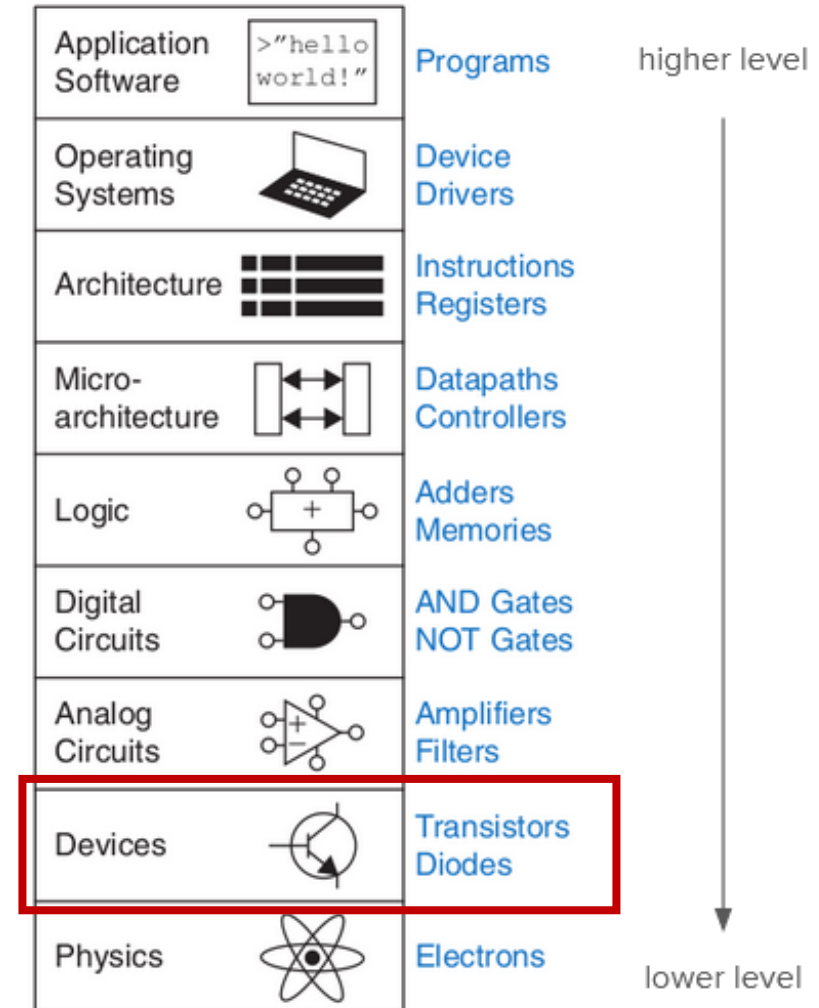
NOT logic gate

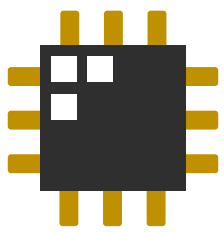


The transistors would be modeled as the switches



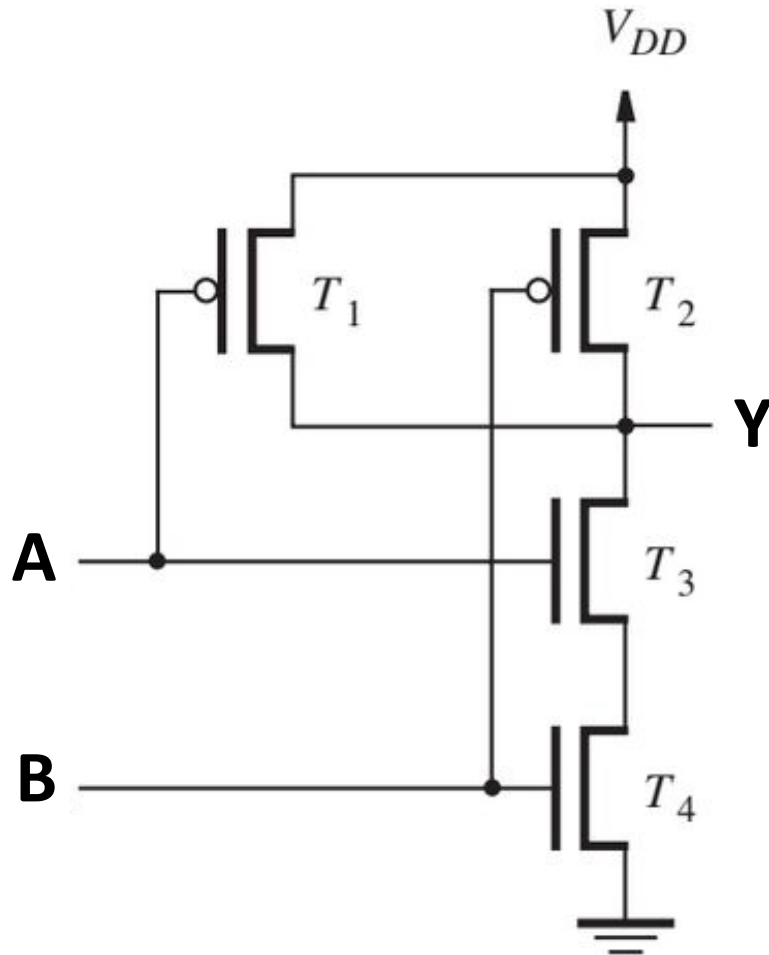
These are transistors!





What are transistors anyway?

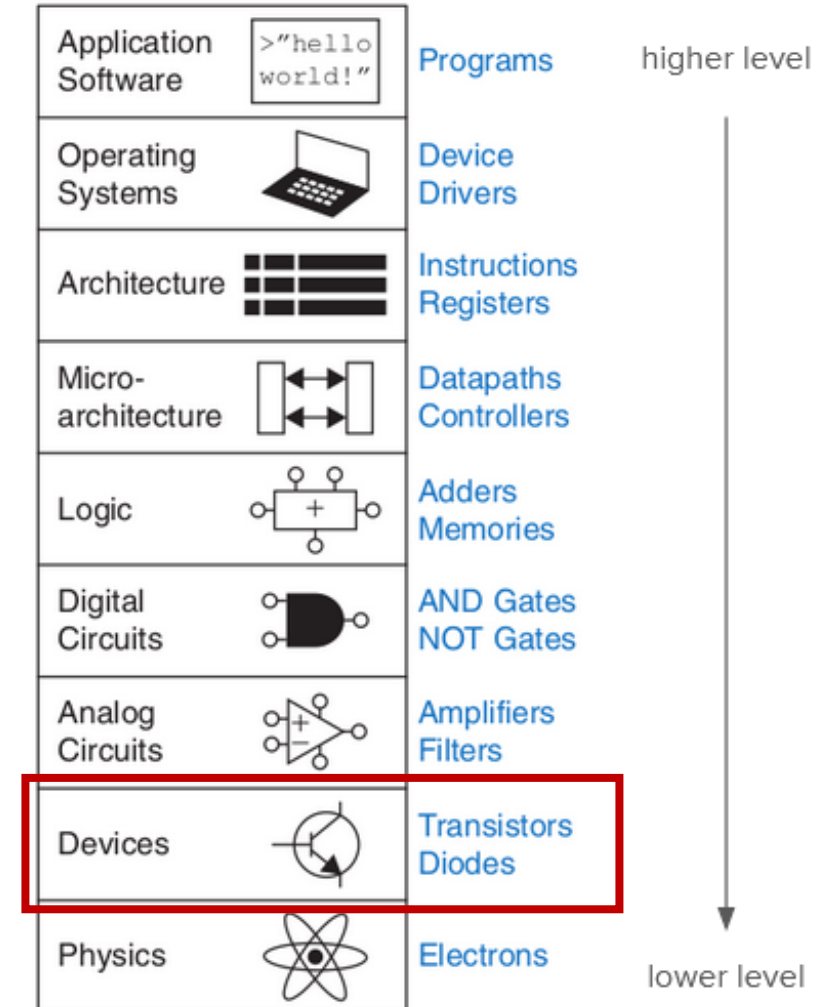
NAND (NOT AND) logic gate

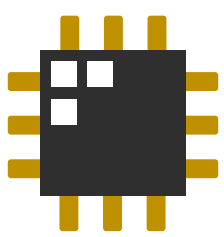


A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

**More complex logic:
more transistors!**

More discussions on transistors will be in EEE 131!





What are transistors anyway?

Transistors are made out of silicon

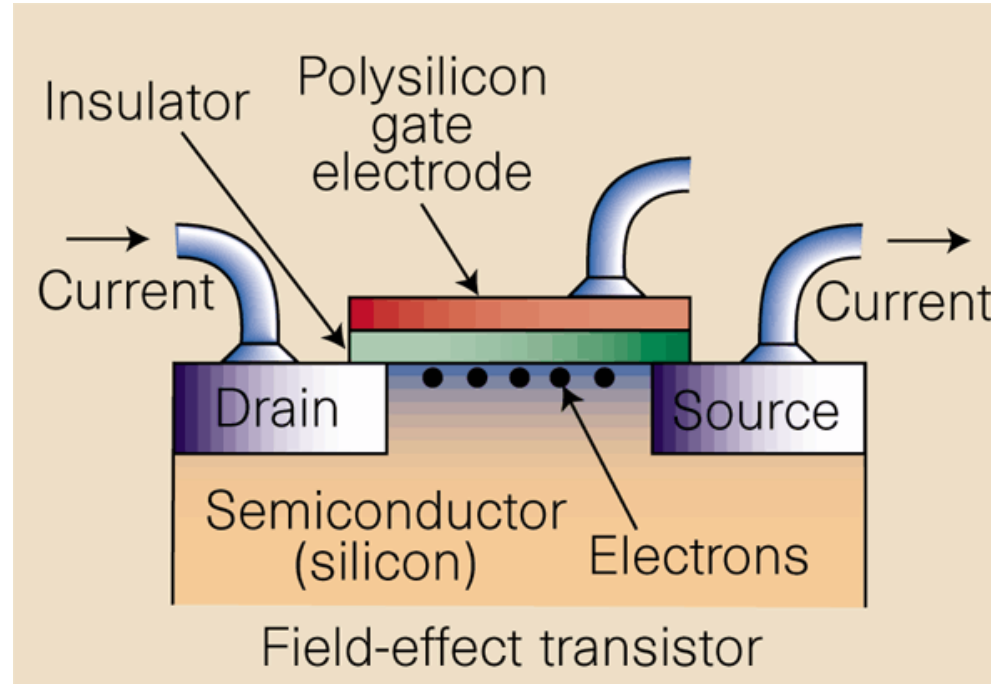
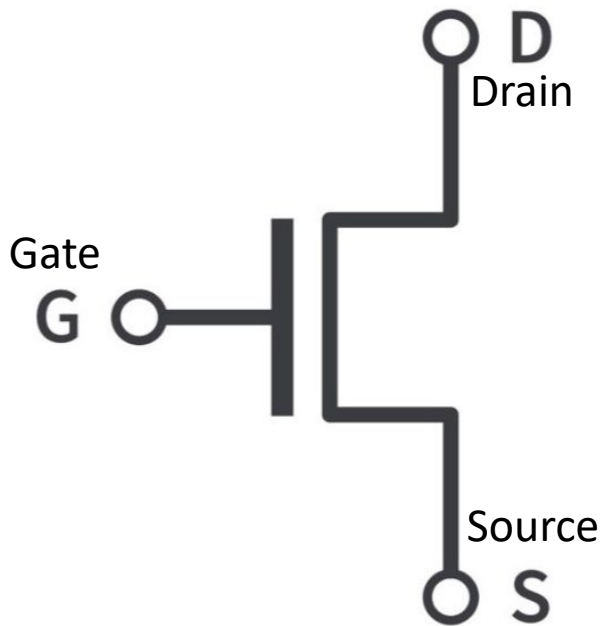


Image credit: <https://www.nature.com/articles/21526>

Transistors are made out of silicon
More complex logic, more silicon area!

More discussions on transistors will be in EEE 131!

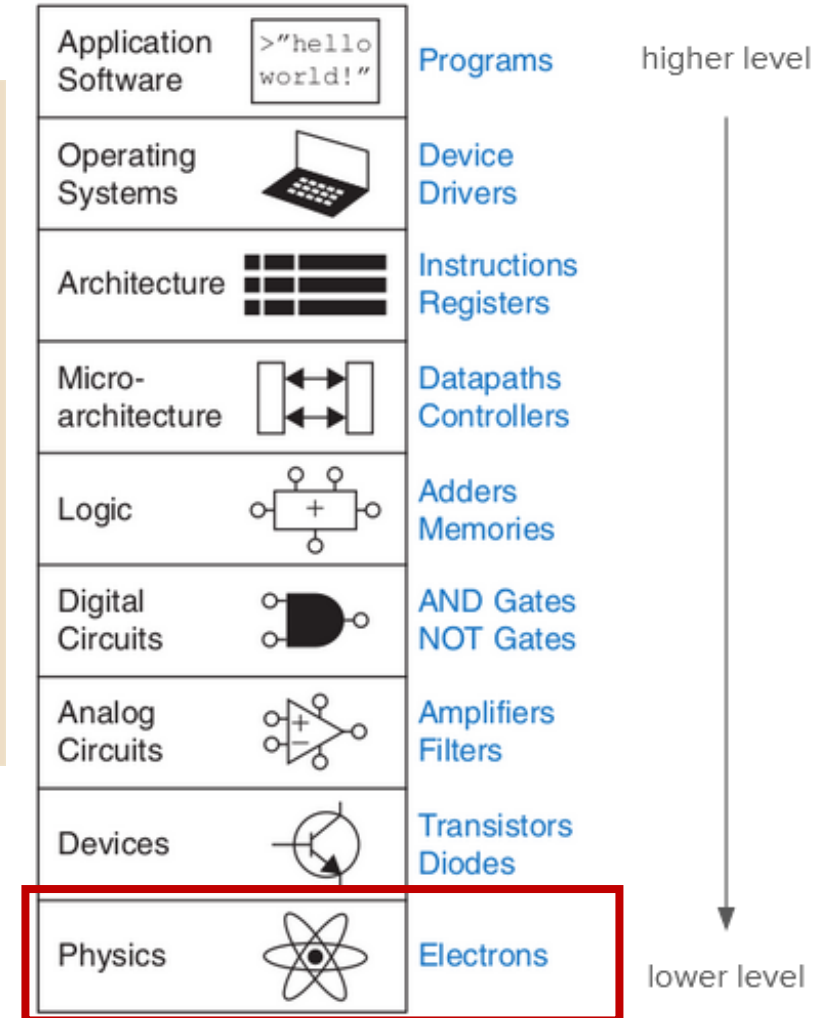
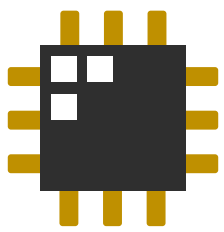
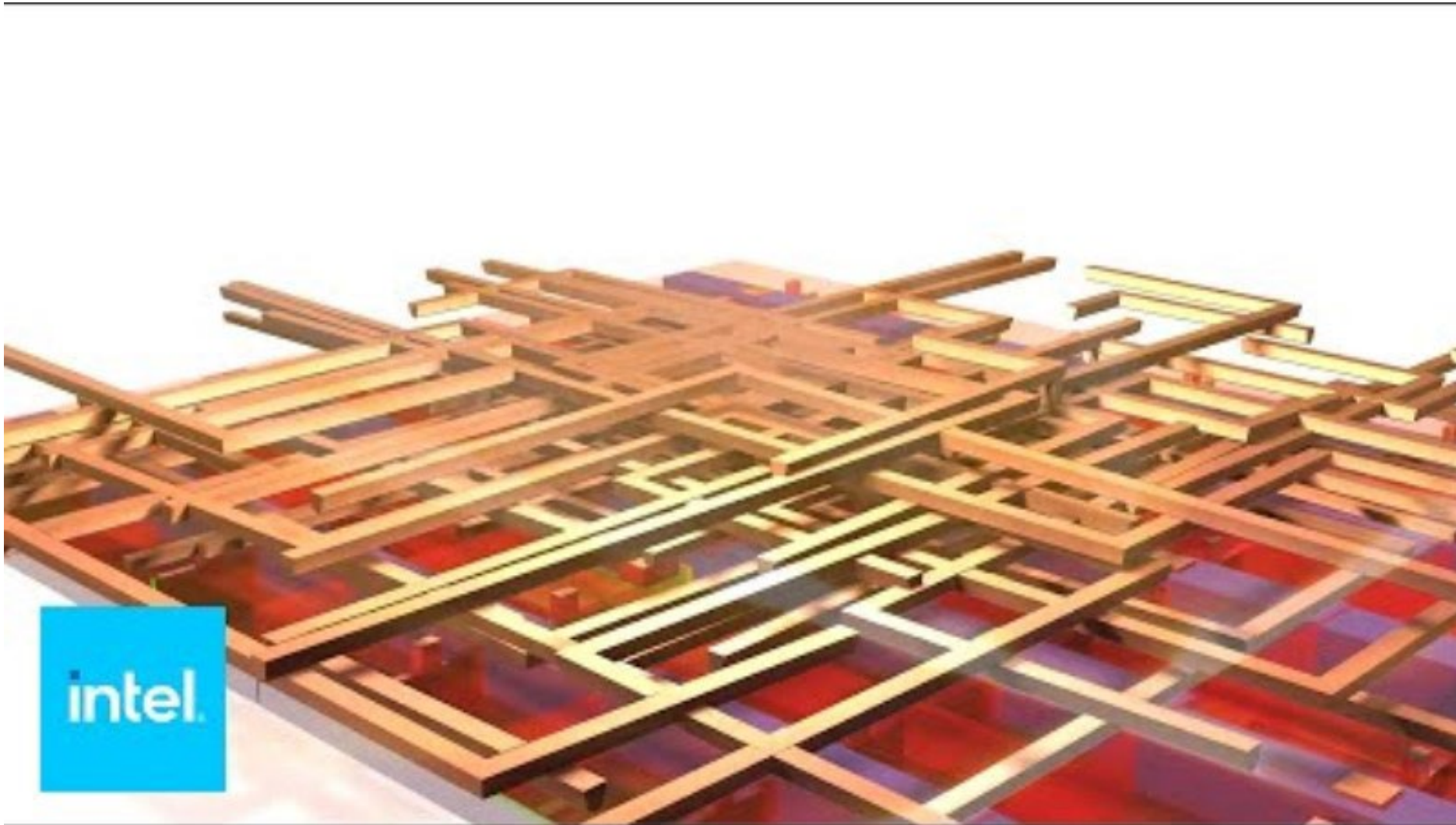


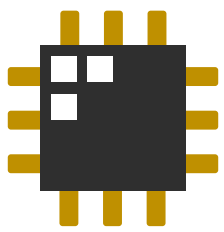
Image credit: *Digital Design and Computer Architecture* (2nd ed.) by Harris, H., Harris, S. (2013).



Fabrication process: Silicon to transistors



Sand (silica) to transistor fabrication process. More about this in EEE 131, CoE 197U!



Processor complexity

Threadripper 3990X Architecture

The Threadripper 3990X comes packing AMD's Zen 2 microarchitecture, but AMD spreads the design across 8 eight-core 'core chiplet die' (CCD), as opposed to four CCD with the 3970X and 3960X and two with the mainstream Ryzen chips.

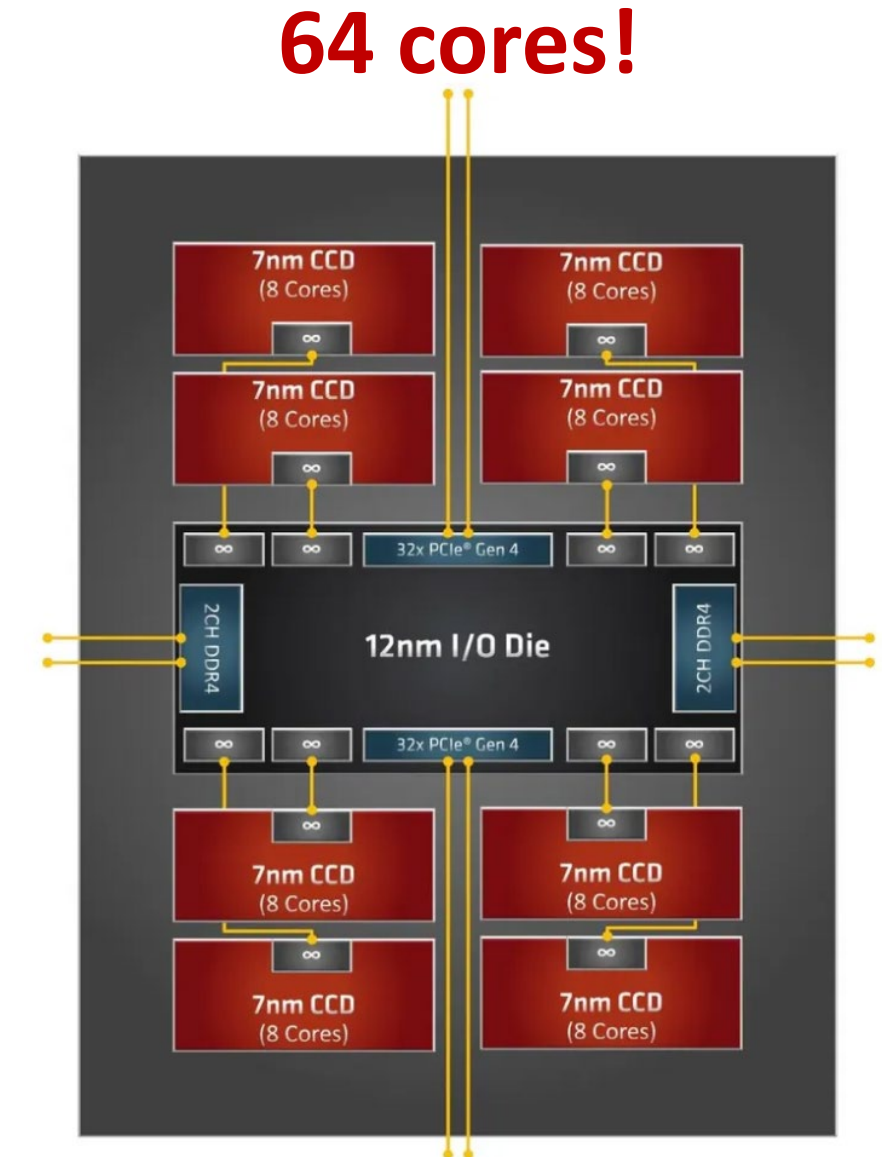
AMD ties the compute chiplets together via the Infinity Fabric to a large central 12nm I/O die (IOD) that houses two 32x PCIe Gen4 controllers and two dual-channel DDR4 memory controllers. However, this IOD is significantly different than the one present in the EPYC Rome 7702P, largely because AMD culled support for eight-channel memory and halved PCIe 4.0 support to 64 lanes.

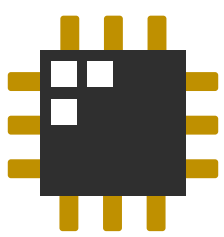
The 3990X's IOD is the same as the one in the 3970X and 3960X, but AMD fused off the four additional Infinity Fabric On-Package (IFOP) units, which are used to communicate with the CCDs, on those models because they weren't needed. As with the previous models, each of the 3990X's die has its own IFOP connection to the IOD.

Each 7nm compute chiplet features ~3.9 billion transistors, while the 12nm I/O die has ~8.34 billion transistors. For the eight-CCD 3990X that yields a total of ~39.54 billion transistors spread out over an amazing ~1008 square millimeters of silicon.

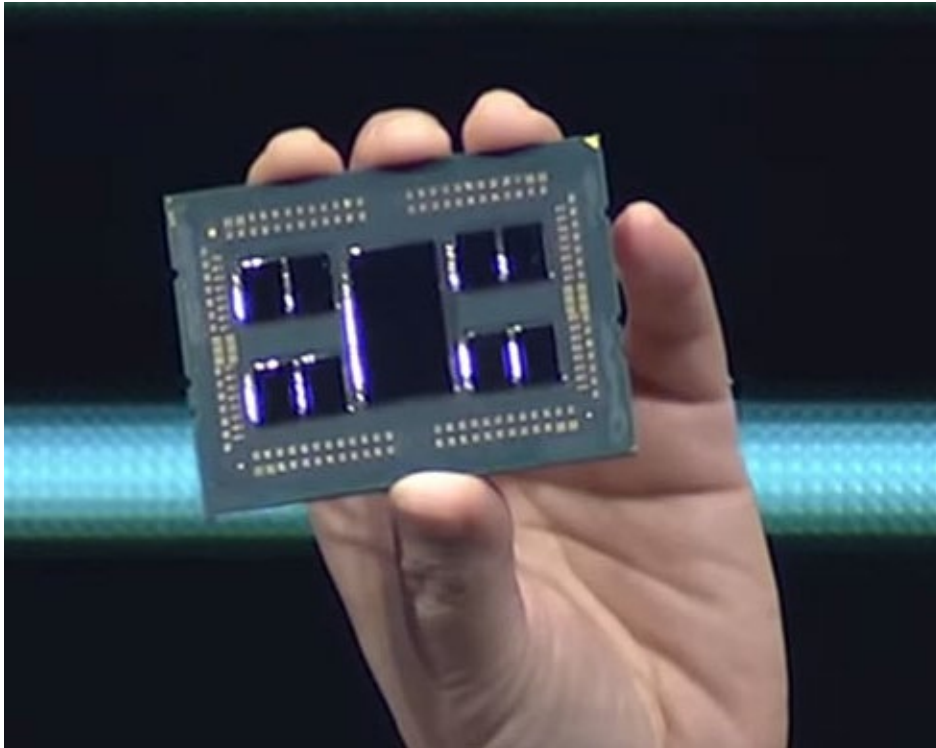
Transistors are made out of silicon. More about this in EEE 131!

<https://www.tomshardware.com/reviews/amd-threadripper-3990x-review>





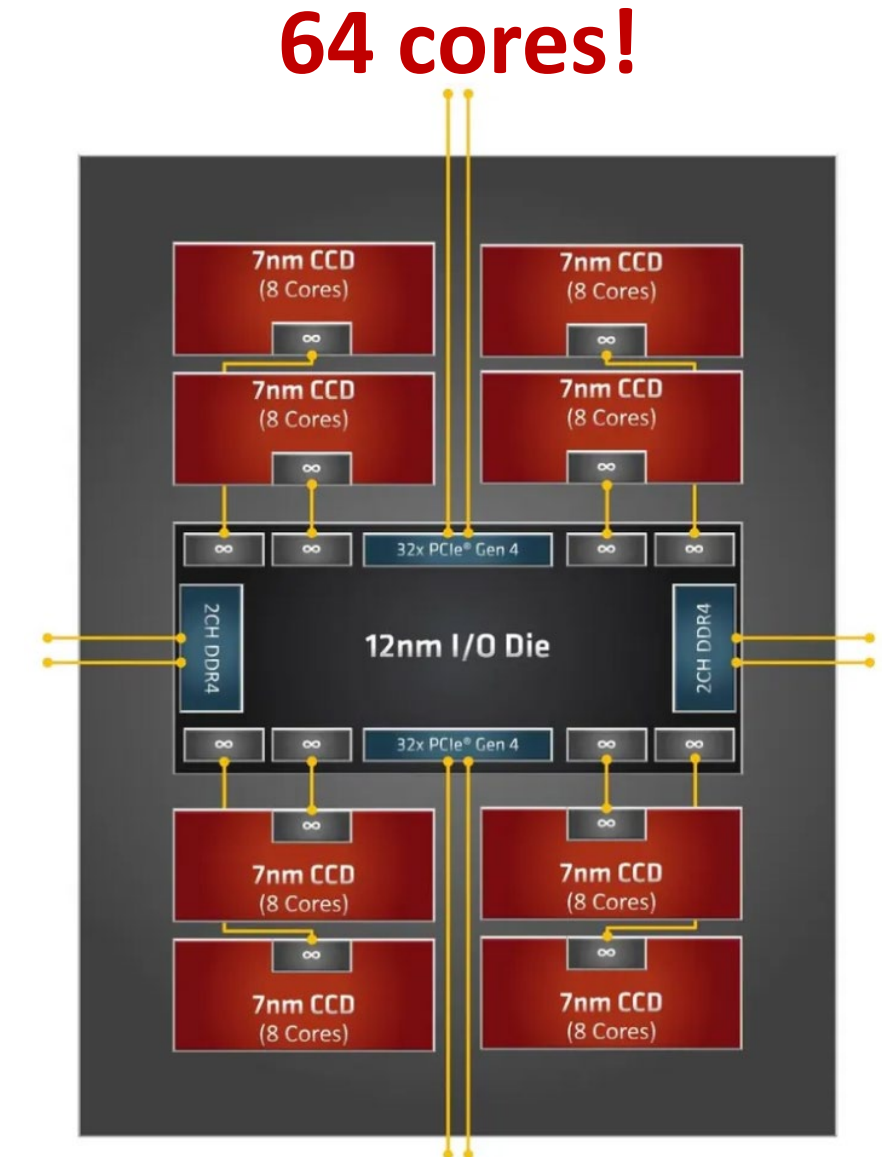
Processor complexity

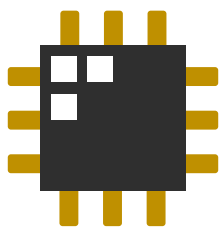


Each 7nm compute chiplet features ~3.9 billion transistors, while the 12nm I/O die has ~8.34 billion transistors. For the eight-CCD 3990X that yields a total of ~39.54 billion transistors spread out over an amazing ~1008 square millimeters of silicon.

Transistors are made out of silicon. More about this in EEE 131!

AMD Threadripper 3990x announcement. CES 2020. January 6, 2020.





End of EEE 113: Computing

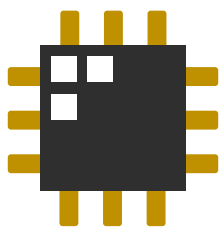
“Every great system begins with a symphony of 0s and 1s.

We turned patterns into meaning; that’s what computing is all about.

See you on the next positive edge.”

-ChatGPT

1	1	1	1	1	1	1	0	0	1	0	0	0	0	1	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	1
1	0	1	1	1	0	1	0	0	0	0	1	1	1	0	1	0	0	1	0	1	1	1	0	1
1	0	1	1	1	0	1	0	1	1	0	0	0	1	1	0	0	0	1	0	1	1	1	0	1
1	0	1	1	1	0	1	0	1	1	1	1	0	0	1	0	1	0	1	0	1	1	1	0	1
1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	1
1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0
0	1	0	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	0	1	1	0	1	0
0	1	0	1	1	0	0	1	0	0	0	0	1	0	1	1	0	1	0	1	1	0	0	0	0
0	0	1	1	0	0	1	0	1	0	0	1	1	0	1	1	0	0	1	1	0	1	0	0	1
1	0	0	1	1	1	0	1	0	1	0	1	1	0	0	0	1	0	1	1	1	1	1	1	1
0	0	0	1	1	1	1	1	0	1	0	1	0	1	1	1	1	1	1	1	0	1	0	1	1
1	1	1	0	1	0	0	1	0	0	1	1	0	0	1	1	0	0	0	0	1	1	1	0	0
1	1	0	0	0	0	1	1	0	1	0	1	0	1	1	1	1	0	1	0	0	0	0	0	1
1	0	0	1	1	0	0	1	0	0	1	0	1	1	0	0	0	0	1	0	0	1	1	1	1
1	0	1	0	0	1	1	0	1	1	1	1	0	0	1	1	1	1	1	1	1	0	1	1	0
0	0	0	0	0	0	0	0	1	1	1	0	0	0	1	0	1	0	0	0	1	0	1	1	1
1	1	1	1	1	1	1	0	0	1	1	1	1	0	1	0	1	0	1	0	1	1	1	0	1
1	0	0	0	0	0	1	0	1	1	0	1	1	0	1	1	1	0	0	0	1	1	0	1	0
1	0	1	1	1	0	1	0	1	1	1	0	1	1	0	1	1	1	1	1	1	1	0	0	1
1	0	1	1	1	0	1	0	1	0	0	1	1	0	1	1	1	1	1	1	0	1	1	1	1
1	0	1	1	1	0	1	0	0	1	1	0	1	1	0	1	0	1	0	1	1	0	0	1	1
1	0	0	0	0	0	1	0	1	1	0	1	0	1	0	0	0	1	0	0	0	0	1	1	1
1	1	1	1	1	1	1	0	0	1	0	0	0	1	1	0	1	1	1	1	1	1	0	0	1



End of EEE 113: Computing

“Every great system begins with a symphony of 0s and 1s.

We turned patterns into meaning; that’s what computing is all about.

See you on the next positive edge.”

-ChatGPT

